University for the Creative Arts

BSBI School of Business and Innovation

DISSERTATION

Automatic Code Base Analysis and Architectural Querying System Using Retrieval-Augmented Generation (RAG) and Large Language Models

Course Title
BSc Computer Science and Digitisation

Name
Barbaros Yesilova / Q1039255

Year
October 2022

ABSTRACT

This study demonstrates that, whilst the integration of Large Language Models (LLMs) into software engineering has led to significant advances in the field of automated code analysis, adoption at an organisational scale is still severely hampered by stringent data privacy risks and the loss of syntactic context in traditional Retrieval-Augmented Generation (RAG) systems. Traditional RAG architectures fragment semantic coherence by typically dividing source code into segments based on arbitrary character lengths, thereby weakening the context window and ultimately leading to poor retrieval performance and LLM 'hallucinations'.

This thesis aims to resolve the 'privacy versus performance' dilemma by proposing a model-agnostic, privacy-focused RAG architecture based on *Abstract Syntax Tree (AST)* parsing. By adopting a hybrid method combining System Development Research and Quantitative Experimental Analysis, a fully functional decision-support prototype has been designed and implemented. The system's information access layer explicitly parses structural nodes (e.g. `FunctionDef`, `ClassDef`) and has been benchmarked against a naive recursive segmentation approach using a 1,000-character length and 200-character overlap on the Flask framework repository.

Experimental results show that both approaches achieved the same hit rate of 86.67 per cent for `@k=3`; however, the AST-aware architecture prioritised target modules more accurately by preventing contextual fragmentation and demonstrated superior performance with a Mean Reciprocal Rank (MRR) of 0.789, compared to the baseline approach's value of 0.678. In the subsequent generation phase, an evaluation using the RAGAS framework and a local '`LLM-as-a-judge`' setup showed that the on-premises Qwen 2.5 (7B) model achieved an impressive 81.3 per cent Faithfulness score and a 79.6 per cent Answer Relevance score. In contrast, the cloud-based Gemini 3.5 Flash model, whilst offering lower inference latency (20.02 s versus 27.35 s), exhibited significant reliance on information beyond the supplied retrieval context due to a tendency to rely on pre-trained assumptions and achieved a Faithfulness score of only 55.1 per cent. These findings provide evidence that treating source code as a syntactic hierarchy rather than plain text can improve code understanding within the evaluated setting. Consequently, the study provides a scalable and experimentally validated roadmap for the development of secure, structure-aware and privacy-compliant AI assistants in software engineering.

CONTENTS

Abstract	2
Contents	3
Acknowledgements	4
Dissertation Thesis	6
Introduction	7
Chapter ONE - Literature Review I	10
Chapter TWO - Literature Review II	19
Chapter THREE - Methodology	27
Chapter FOUR - Findings, Analysis and Discussion	32
Concluding Remarks	50
Bibliography	53
Appendix	58

ACKNOWLEDGEMENTS

I would like to express my heartfelt thanks to my supervisor, Dr Eng. Loubna Ali, whose knowledge, experience and support have guided me throughout the process of writing this thesis. Her guiding feedback and constructive criticism during the stages of clarifying the problem definition, refining the experimental design and interpreting the results from a critical perspective have directly contributed to the scientific depth and integrity of this thesis.

I would like to extend my appreciation to the members of the exam board whose advice enhanced this piece of work, to all my academic advisors for their scholarly contributions, to the officials of the institutions and organizations who went out of their way to create a technical environment for me, and to my colleagues.

I am indebted to my family and friends whose moral support was always there during this tough process for their patience and understanding. It wouldn't have been possible without their total support.

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people's texts I clearly declare it through the good use of references following academic ethics.

(In the case that it is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the program).

Name and Surname (Capital letters):

BARBAROS YESILOVA

.....

Date: ....**31**.../..**08**.../.**2026**

DISSERTATION THESIS

INTRODUCTION

Background and Context

The rapid integration of Artificial Intelligence (AI) into Software Engineering (SE) has transformed modern software development. With the increasing complexity of modern systems, which have many millions of lines of code and dynamically changing frameworks, developers find it difficult to go through the code repository when they are new to it. Whilst traditional static analysis tools can navigate code structures using symbol tables, they lack the natural language reasoning capabilities required to answer complex architectural queries.

Whilst modern Large Language Models (LLMs) demonstrate extraordinary generative capabilities, their ability to directly analyse enterprise-scale data repositories is limited due to context window constraints, quadratic computational complexity and hallucinations. Retrieval-Augmented Generation (RAG) offers a solution to these problems by grounding outputs in external information sources. However, traditional RAG systems, originally designed for unstructured text, frequently fail to preserve the rigid hierarchical and syntactic integrity inherent in source code. Consequently, the development of intelligent systems capable of 'structure-aware code retrieval' has become an urgent necessity.

Triggering Factors and Rationale

This research stems from a critical technological and industrial dilemma. Standard RAG applications segment source code using simple, character-based sliding windows (e.g. fixed 1,000-character chunks). This arbitrary 'flat' segmentation leads to 'context fragmentation' and 'information dilution' by splitting class definitions and function scopes. As a result, the LLM's semantic reasoning ability is weakened, leading to erroneous architectural assessments.

At the same time, enterprise software development processes require strict intellectual property protection. Sending proprietary code to cloud-based LLM APIs carries unacceptable risks of data leakage. On the other hand, whilst fully isolated on-premises Small Language Models (SLMs) offer complete security, their low parameter scales raise concerns regarding analytical depth.

This thesis addresses the aforementioned 'privacy and performance' dilemma. A model-agnostic, AST-aware RAG architecture is proposed, which runs locally on consumer hardware and is based on Python's Abstract Syntax Tree (AST) module. This approach enables organisations to execute structurally-based architectural queries without compromising code confidentiality or incurring ongoing cloud API costs.

Purpose and Objectives of the Research

The general objective of this thesis is to design, develop, and evaluate a privacy-oriented, model-agnostic RAG architecture using AST parsing techniques. In order to accomplish the above general objective, four different objectives have been formulated.

Objective 1: To design a syntax-aware code ingestion engine that uses FunctionDef, ClassDef, and AsyncFunctionDef nodes parsing using Python's AST library.

Objective 2: To create an interactive, model-agnostic decision-support prototype capable of running both on-premises (Qwen 2.5 7B via Ollama) and cloud-based models (Gemini 3.5 Flash), using Streamlit, LangChain and ChromaDB.

Objective 3: To quantitatively compare the AST-aware code ingestion approach with the traditional character-based recursive tokenisation approach (1,000 characters, 200 overlap). This comparison will be conducted using a 15-question architectural question dataset obtained from the Flask repository, based on the Hit Rate @k=3 and Mean Reciprocal Rank (MRR) metrics.

Objective 4: To evaluate Answer Faithfulness, Answer Relevance, and inference latency using the RAGAS framework with a privacy-oriented local LLM-as-a-judge pipeline based on Qwen 2.5 7B.

Overview of the Research Methodology

This research adopts a hybrid methodology combining System Development Research (SDR) and Quantitative Experimental Analysis methods. The SDR phase guides the modular design and deployment of the functional RAG system using Python 3.11, LangChain, ChromaDB, Ollama and Streamlit.

- In the experimental phase, the architecture is tested against a baseline using the Flask framework repository. Two distinct vector databases are created: an AST-aware experimental database and a baseline database generated via standard recursive text segmentation. Both utilise the same nomic-embed-text embeddings and search parameters (k=3).
- Retrieval accuracy is evaluated using Hit Rate (@3) and MRR on a dataset of 15 queries. Output quality is measured via the RAGAS framework using a local instance of ChatOllama (Qwen 2.5 7B), which runs as an automated evaluator model on an Apple Silicon (M4 Pro) host.

Dissertation Structure

The thesis is systematically divided into four main chapters and a conclusions section:

- Chapter One (Literature Review I): Examines the historical development of static code analysis, LLM context constraints, vector-based Information Retrieval, and vendor lock-in scenarios.
- Chapter Two (Literature Review II): Analyses current AI-powered code review tools, compares structural AST parsing with lexical tokenisation, reviews RAGAS metrics and identifies research gaps.
- Chapter Three (Methodology): Details the SDR framework, system architecture layers, dataset preparation and experimental comparison protocols.

- Chapter Four (Findings, Analysis and Discussion): Presents user interface results, quantitative performance metrics (Hit Rate and MRR), RAGAS evaluation metrics and an analysis of architectural designs.

- Concluding Remarks: Synthesises the findings, summarises the contributions to software engineering applications, highlights technological limitations, and suggests future work on GraphRAGs with multilingual support.

CHAPTER ONE – LITERATURE REVIEW I

1.1 Evolution of Code Comprehension and Static Analysis

Developers' methods for understanding code have evolved considerably, keeping pace with the increasing size and complexity of software. In the past, code comprehension and static analysis began with straightforward text-based techniques and gradually advanced to more organized and semantic methods using program representations, such as Abstract Syntax Trees (ASTs).

Early and Classical Methods of Code Comprehension

At the very beginning of software development, developers tend to use primitive search tools like grep. This command line utility searches the source files for keywords by searching lines of matching regular expressions in plain text. It works well enough for small to medium-sized projects, but it does not perform well for large, complex codebases because it relies solely on lexical matching and lacks any syntactic or semantic understanding of the code.

The usage of IDEs for improving code comprehension involves making use of indexers that go through all the code to generate symbol tables and indexes to make navigation and search fast. These indexes are used by IDEs to provide features such as “go to definition,” “find all references,” and auto-completion of code. However this does not involve any deep understanding of semantics but only simple token indexing and matching.

Nevertheless, both grep and IDE indexing methods face scalability issues in large and heterogeneous projects. For example, in massive systems like Eclipse or Mozilla, which contain tens of thousands of methods, traditional keyword searches can produce a large number of irrelevant results or require the examination of numerous code fragments. Research has shown that, in order to effectively identify concepts, programmers must combine these searches with mental models or ontological components representing domain concepts by focusing on small subsets of relevant code rather than the entire code base (Petrenko, Rajlich, and Vanciu, 2008).

Importance of Abstract Syntax Trees (AST)

The invention of the abstract syntax trees (ASTs) marks a major breakthrough in understanding code and performing static analysis on the source code. The abstract syntax trees represent the structure of the code hierarchically, removing all the extraneous elements like comments and white spaces but retaining the grammatical relationships between the constructs of the programming language.

ASTs play an integral role in classical compiler architecture by aiding in parsing the input program and supporting the process of further compilation stages like semantic analysis, optimization, and code generation. Moreover, developers make use of ASTs to carry out tasks like static analysis and program transformations. In addition, due to the structure of the AST, it makes possible to do even more sophisticated analyses, for instance, find code clones not only based on textual similarity but structural as well.

Nevertheless, using ASTs poses some difficulties when used for large-scale programs. The first problem is that ASTs for big enough code fragments turn out to be quite big and complex to traverse and analyze. The direct naive approach to analyzing the whole AST with usual algorithms results in high costs and, moreover, might cause some problems, for example, the problem of gradient vanishing when using ASTs in neural networks (Zhang et al., 2019; Liang and Ai, 2021).

For example, neural models such as ASTNN split large ASTs into smaller parts and encode them sequentially, thereby better capturing lexical and syntactic information for tasks such as source code classification and clone detection (Zhang et al., 2019). Similarly, representing code fragments as sets of compositional paths through ASTs enables a more robust classification than traditional separate embedding approaches (Liang and Ai, 2021).

Static Analysis and Challenges of Modern Complex Software

By definition, static analysis tools work by analyzing source code or an intermediary representation like an abstract syntax tree (AST). By studying syntax and semantics of the code, static analysis tools find bugs, security problems, and design patterns. However, with the increase in the size of software and its complexity, especially in case of incorporation of frameworks, reflection and dynamics, traditional static analysis becomes increasingly limited.

For example, reflection mechanisms based on the Java Reflection API may bring up behaviors in code which are hard to predict by static analysis. Reflection can result in unsoundness of static analysis and the need to make unsound assumptions to predict runtime behavior of code. Research has found that the majority of Java projects heavily use reflection and therefore static analysis is rather difficult for these projects without heuristics and approximation (Landman, Serebrenik, and Vinju, 2017).

In addition to this, the contemporary software architecture is becoming heterogenous and brings high-level abstractions like microservices, data entities, controllers, and endpoints which are used across different languages and frameworks. It exceeds capabilities of traditional static analysis tools to analyze low-level language constructs separately. The modern approach to static analysis is advocated by research to transform source code into intermediary representations like language-agnostic abstract syntax tree and extract meaningful high-level abstractions using generalized parsers.

Summary of Limitations and Bottlenecks in Large and Complex Projects

Summary of Limitations and Bottlenecks in Large and Complex Projects

Technical Approach	Key Technical Limitation	Primary Bottleneck in Large Scale	Relevant Citation
Text-Based (grep)	Relies solely on lexical matching; lacks semantic understanding.	Requires domain knowledge and selective inspection of fragments.	Petrenko et al. (2008)
IDE Indexing	Relies mostly on token-level indexing; lacks deep syntactic understanding.	Scalability bottlenecks arise in large or heterogeneous projects.	—
Classical ASTs	Faces computational and representational challenges as trees grow.	ASTs become massive and complex, affecting traditional and ML models.	Zhang et al. (2019); Liang & Ai (2021)
Dynamic Features	Reflection and dynamic language features limit analysis precision.	Requires pragmatic unsound assumptions with limited guarantees.	Landman et al. (2017)
Modern Architectures	Exceeds capacity to analyze low-level language constructs in isolation.	Complex multi-language and component-based designs.	Schiewe et al. (2022)

As a whole, the development of code understanding and static code analysis is a path from simple text search to increasingly more advanced, structure-aware and semantic techniques, developed in response to the growing scale and diversity of modern-day software projects.

1.2 Large Language Models (LLMs) in Software Engineering

There has been some significant success in the field of software engineering through the use of Large Language Models (LLMs). These models work using deep neural networks and have been trained on large datasets which include both natural languages and source code, making it possible for them to understand, generate and reason about code. This section offers an in-depth look at what the models can do, how LLMs came to exist in the code ecosystem, what is the major technical challenge faced by the models, called the Context Window problem, and why it is not possible to train the models on large code repositories theoretically.

Capabilities of LLMs in Code Writing and Analysis

Large Language Models such as ChatGPT-5.6, Claude Opus 5 and Google Gemini 3.7 have demonstrated proficiency in performing several vital tasks associated with software engineering.

- Code Generation: Large Language Models have the capacity to generate code blocks, functions, or even complete modules from natural language commands. Their capabilities include not only basic code

generation but also other more difficult tasks; however, their effectiveness decreases with the complexity and length of the code (Shehab et al., 2024; Zhang et al., 2024).

- Fault Localization and Debugging: LLMs can detect faults in the code through logic analysis and error context analysis. For example, ChatGPT-4 has demonstrated its superiority over earlier fault localization methods with the help of function-level code context (Wu et al., 2023); but their effectiveness drastically reduces as the code context increases (class level or beyond).

- Code Summarization and Documentation: Producing understandable and concise summaries of code enables programmers to quickly comprehend large and complex codes. LLMs using advanced prompting technology produce better code summaries using syntax and structural analysis, though there are still difficulties in accurately identifying the specific requirements of the task (Fang et al., 2025).

- Code Design and Localization of Design Errors: Apart from line-level errors, LLMs are used more frequently in solving high-level design issues such as modularity and complexity issues, through the use of multi-agents in transforming technical outputs into LLM prompts (Batole et al., 2025).

Despite these advancements, the generated code is currently not suitable for production purposes but only for prototyping and learning due to its incomplete nature (Jin et al., 2024).

The Rise of LLMs in the Code Ecosystem

The growing integration of LLMs into software engineering workflows is driven by their broad utility.

- Automating routine coding tasks reduces the developer burden.

- Natural language-based code generation brings programming to everyone who is not an expert.

- Helping in code review, testing, and identifying bugs speeds up the entire process.

- Bettering the model structure and training data makes the system more accurate and applicable.

CODEAGENT framework goes further with combining LLMs and external software engineering tools to address sophisticated repository-based problems (Zhang et al., 2024).

Nevertheless, the deployment of LLMs at scale involves critical engineering challenges, among which the Context Window limitation is the most pronounced.

The Context Window Problem: Major Technical Constraint

The Context Window, sometimes called the attention window or context length, refers to the maximum length of input tokens (words, code tokens, etc.) an LLM can process simultaneously during inference. This length constrains the amount of code and related context (comments, documentation, and error logs) that models can consider when generating or analyzing code.

Typical current models have a context window on the order of a few thousand tokens (e.g., 2 K–8 K tokens).

When codebases exceed this length, only partial code snippets can be input at once, forcing the model to ignore the broader context or split the problem.

The practical implications are observed directly in performance drops when the code context length is increased beyond the function level to the class level or larger units, as demonstrated for ChatGPT-4's fault localization accuracy (Wu et al., 2023).

Why Very Large Codebases Can not Be Fed Directly into LLMs?

It is due to the underlying mechanism in these transformer-based LLMs:

- Quadratic Scaling: Attention processes, which form the heart of transformer models, exhibit quadratic scaling behavior with respect to the number of input tokens. This means that an increase by twice the number of input tokens would roughly quadruple the cost of computation and storage.

- Token Limit Hard Caps: Models are trained and deployed with fixed token limits owing to hardware and architectural constraints. Actual software applications may have tens to hundreds of thousands of lines of code in real-world scenarios, exceeding all these boundaries.

- Information Dilution: Even if larger context windows are feasible, using large volumes of raw code may overwhelm the model with extraneous information and lead to information dilution. Effective input usually requires careful context selection and a summarization.

Hence, strategies have emerged to address this issue.

- Context engineering: Systematic optimization and compression of inputs to maximize relevant information within fixed-length contexts (Mei et al., 2025).

- Agent-based approaches: Multi-agent architectures where the agents perform processing, summarization, and prompt-generation to direct the LLMs to reason about certain portions of the codebase without having the whole codebase fed to them (Zhang et al., 2024; Batole et al., 2025).

- Planning and Decomposition Approaches: Enabling LLMs to plan and decompose complicated tasks into simpler tasks, thus generating code in an incremental manner constrained by the context window (Jiang et al., 2024).

Conclusion

In conclusion, LLMs have become highly effective tools in software engineering that can write, debug, and summarize code. The increasing popularity of LLMs in the field of coding is due to considerable advances in natural language understanding and code generation. However, the fixed and relatively limited Context Window remains a fundamental bottleneck, constraining the direct feeding of enormous code repositories into these models. This limitation arises from the quadratic computational complexity of transformer attention and fixed token limits in LLMs, making the processing of large-scale codebases theoretically and practically unfeasible. To bypass this, approaches such as context engineering, multi-agent tool integration, and task planning have been actively developed to enable effective and scalable code analysis and generation without overwhelming the model's context capacity (Wu et al., 2023; Jiang et al., 2024; Zhang et al., 2024; Batole et al., 2025; Mei et al., 2025).

1.3 Retrieval Augmented Generation (RAG)

Retrieval Augmented Generation (RAG) is a groundbreaking framework aimed at improving the capabilities and accuracy of LLMs through the incorporation of retrieval of external knowledge sources and generative text creation capability into one system (Kiran, 2025). In contrast to LLMs that have problems with cut off of knowledge and hallucinations, the RAG systems generate output based on the retrieval of documents or snippets from knowledge sources during runtime (James, Trovati and Bolton, 2025).

Core Concept and Workflow of RAG

To summarize, RAG works on the principle of incorporating a retrieval function within the generative architecture of the pipeline. The first step involves searching for the appropriate document chunks through an external knowledge source based on the question asked by the user. The selected context is added to the list of inputs to generate an output response.

This hybrid approach allows RAG systems to:

- Mitigate hallucinations by grounding generation in real knowledge
- Access up-to-date or domain-specific information beyond the LLM's training data
- Improving accuracy and relevance, especially in specialized domains such as healthcare, finance, and academic research (James, Trovati and Bolton, 2025; Kiran, 2025).

Information Retrieval Theory, Vector Embeddings, and Semantic Search in RAG

Theory of Information Retrieval and Retrieval Approaches in RAG

The theory of information retrieval is the basis of the RAGs' retrieval process in relation to finding and ranking relevant documents in response to a particular query. There are lexical and semantic approaches to retrieval used by RAG to enhance the relevance of found documents.

- Lexical Retrieval: Traditional IR techniques include keyword searching and sparse vector space approaches (TF-IDF, BM25).
- Semantic Retrieval: Nowadays, RAG uses dense vector representations that allow capturing the semantic content of documents beyond the mere lexical match (Sawarkar, Mangal and Solanki, 2024; Kiran, 2025).

Vector Embeddings

Vector embeddings refer to the process of representing textual data as a continuous vector in a high-dimensional space. They represent semantic information and help find similarities between texts efficiently.

- Pretrained models such as sentence transformers or OpenAI embeddings are used to convert text chunks (documents, paragraphs) and queries to embedding vectors.
- Similarities between the embedding vectors of a query and a document are measured using similarity metrics, especially the cosine similarity to find similar semantic meanings of documents (Tanyildiz, Ayvaz and Amasyali, 2024; Pokhrel, C and Shah, 2025).

Semantic Search Using Cosine Similarity

Cosine similarity refers to measuring the angle between two vectors and, hence, capturing their semantic similarities irrespective of their magnitudes.

- A high score of cosine similarity means that there is more relevance contextually between the two vectors.
- Cosine similarity can be used for the purpose of semantic search for conceptually similar documents instead of just matching keywords (Pokhrel, C and Shah, 2025).

Vector Databases Vs. Traditional Relational Databases

Vector Databases: ChromaDB and FAISS

Vector databases, such as ChromaDB and FAISS, are specialized for storing and querying high-dimensional embedding vectors.

- FAISS (Facebook AI Similarity Search) is a highly optimized library that supports efficient nearest-neighbor search using various indexing methods, such as Hierarchical Navigable Small Worlds (HNSW), improving retrieval speed and accuracy in large-scale vector data (Tanyildiz, Ayvaz and Amasyali, 2024).
- ChromaDB offers composable vector storage with semantic search capability and is built to handle dynamic, large-scale embedding vectors with support for hybrid retrieval methods (Pokhrel, C and Shah, 2025).

Theoretical Differences from Traditional Relational Databases

Concept	Vector Databases (ChromaDB, FAISS)	Traditional Relational Databases
Data Representation	Stores data as high-dimensional vectors representing semantics	Stores structured data in tables with rows and columns (relational model)
Query Mechanism	Retrieves by nearest neighbor search in vector space using similarity metrics (e.g., cosine similarity)	Uses structured query languages (SQL) for exact or range queries based on key values and relations
Indexing	Employs approximate nearest neighbor (ANN) indexing for fast similarity search (e.g., HNSW, IVF)	Uses B-tree or hash indexes to optimize structured data lookup
Flexibility	Supports unstructured text, images, or other modality embeddings	Best suited for structured, schema-defined data
Purpose	Semantic retrieval and fuzzy matching for AI and NLP applications	Transactional data processing, consistency-critical operations
Performance Focus	Speed and scalability of similarity search in very high-dimensional spaces	Efficient discrete data retrieval, data integrity

These differences reflect the unique requirements for retrieval in RAG systems, where semantic relevance, rather than exact matches, drives effective information retrieval (Tanyildiz, Ayvaz and Amasyali, 2024; Kiran, 2025).

Summary

RAG architectures emerged to overcome the knowledge limitations of LLMs by incorporating Information Retrieval (IR) techniques based on vector embeddings and semantic search. The text and queries are embedded into vector spaces, enabling retrieval via similarity metrics, such as cosine similarity, which identifies semantically relevant documents. Specialized vector databases, such as FAISS and ChromaDB, form the backbone of RAG retrieval components, providing efficient high-dimensional similarity searches through indexing methods suited for unstructured and embedding data. These systems differ fundamentally from traditional relational databases because they focus on approximate nearest neighbor searches in semantic spaces rather than exact key-value lookups, thereby revolutionizing knowledge retrieval for language models (Tanyildiz, Ayvaz and Amasyali, 2024; Kiran, 2025; Pokhrel, C and Shah, 2025).

1.4 Vendor Lock-in and Model-Agnostic Software Design

VVendor lock-in in software engineering can be defined as the risk and dependency associated with being tied to a certain provider's technology environment, including its tools and platforms. This tie increases the dependence greatly, making switching of the providers or migration of a certain solution impossible without considerable costs and effort spent. In terms of cloud-based software engineering and adoption of technology, the vendor lock-in issue can be regarded as the key obstacle due to lack of interoperability and standardization of different solutions.

Risks of Vendor Lock-In in Software Engineering

Vendor lock-in poses some critical risks.

- Reduced flexibility: Organizations will be limited to working with the environment provided by a single vendor and thus unable to take advantage of alternative technologies.
- Higher cost of switching: Moving from the current provider may require considerable expenses related to migration of data or redevelopment of a system.
- Dependency on proprietary standards: The vendor may employ proprietary APIs or other proprietary components that make moving more complicated.
- Inhibited innovation: Being constrained by one vendor may limit innovation because there will be less exposure to innovative or specialized tools provided by other vendors.
- Vendor failure risk or strategy change: Should the vendor decide to change their service policy, cease providing support for the service, or close down, the clients locked into using the vendor's service will encounter considerable problems.

An analytical study assessing vendor lock-in from a business perspective demonstrates how, as more computing resources are transferred from on-site to cloud environments, these difficulties become even more pronounced due to the increasing number of proprietary standards that hinder interoperability and application portability (Opara-Martins, Sahandi and Tian, 2016).

Significance of Abstraction Layers in System Architecture

To avoid vendor lock-in and enhance flexibility, a use of abstraction layers in system architecture is the usual and effective approach. An abstraction layer is an intermediary component that makes the higher application layer independent of the platform specifics. It gives a few benefits:

- Vendor independence: Standardized interfaces make the application vendor-independent and help replace or integrate other services.
- Ease of integration: The programmer works with abstractions rather than multiple vendor APIs, which saves efforts.
- Portability: The application is modular, so it can be easily migrated to another framework/infrastructure.
- Maintenance and scalability: There is a clean separation of concerns in the architecture that allows evolution of some system components independently.

In certified concurrent programming and kernel development, for example, modular layered abstractions have been demonstrated to refine complex software components by separating low-level machine models from data abstractions and concurrency reasoning, thus achieving safe composability while reducing the software verification complexity (Kim et al., 2017). This layered methodology parallels the vendor-agnostic design aims of application development.

Model-Agnostic Software Design with Frameworks like LangChain and LlamaIndex

Modern AI and LLM-based application development exemplifies the need for abstraction to avoid vendor lock-in with specific LLM providers. Frameworks such as ***LangChain*** and ***LlamaIndex*** play crucial theoretical and practical roles in achieving model-agnostic software design by providing modular, extensible architectures that abstract the underlying large language model engines.

The main idea behind the creation of ***LangChain*** is developing a modular platform that facilitates the process of creating, deploying, and producing applications based on LLMs. Thus, it provides developers with such reusable building blocks as chains (pipelines) and abstractions for chat models, retrieval-augmented generation, and secure interaction with APIs. Moreover, the use of the framework helps separate application logic from the particular vendor of LLM, making it possible to use different models or combining several LLMs easily. This way, the danger of vendor lock-in can be minimized due to flexibility and interoperability of LangChain across many AI services (Topsakal and Akinci, 2023; Mavroudis, 2024, 2025; P, 2024).

LlamaIndex aims at data indexing and retrieval in order to facilitate working with LLMs and provides an abstraction over data sources and vector databases. In terms of architecture, LlamaIndex emphasizes scalable and model-agnostic retrieval that allows plugging in other embedding models and LLMs without altering the application code. This feature makes it easy to implement changes related to new models (Malviya, Javalkar and Malviya, 2024).

Thus, both frameworks are used to create retrieval-augmented generation pipelines that can be customized, modular, and decoupled from the backend implementation of any LLM.

Theoretically speaking, LangChain and LlamaIndex can be viewed as the most important abstraction layers of LLM-based software applications, which have several functions that comply with the principles of software engineering in order to ensure that the system will not fall into vendor lock-in.

- Encapsulation of vendor-specific APIs: They give uniform access points that conceal vendor-specific quirks and reveal consistent programming abstractions.

- Interchangeability and extensibility: The modularity allows one to plug in various LLM providers or even one's own model without any configuration changes in order to experiment and be future-proof.
- Ensuring the design of the system is secure and scalable: They help to solve security issues and performance of the system by standardizing its components, including prompt templates, memory management, and chain orchestration among others.
- Increasing developer productivity: The presence of high-level abstractions speeds up the development process from prototyping to deployment and also makes it possible to use multi-vendor strategy (Topsakal and Akinci, 2023; Akkiraju et al., 2024; Mavroudis, 2025).

In other words, LangChain and LlamaIndex are examples of the practical implementation of the abstraction layers that comply with the software engineering principles in order to avoid vendor lock-in problems.

Summary

Vendor lock-in is associated with significant risks for software engineering because it reduces flexibility and raises the cost due to using proprietary cloud and AI services. Applying the abstraction layers to design the system architecture is important in order to make the system independent from any vendor. LangChain and LlamaIndex are the examples of such abstraction framework in the field of LLMs that ensure the modularity and standardized interface for model-agnostic application development (Opara-Martins, Sahandi and Tian, 2016; Malviya, Javalkar and Malviya, 2024; Mavroudis, 2025).

CHAPTER TWO – LITERATURE REVIEW II

2.1 Critical Analysis of Current AI-Supported Code Q&A Tools: Architectures, Methods, and Performance

The integration of Artificial Intelligence (AI) has increasingly become common practice in software development, including in code Q&A and code analysis applications on platforms like GitHub. This integration has fundamentally transformed coding workflows and knowledge dissemination in programming domains. A critical review of AI-assisted code Q&A and analysis systems reveals a variety of architectural paradigms, methodological approaches, and varying degrees of success in improving developer productivity and code quality.

One of the architectures employs large language models coupled with Integrated Development Environment (IDE) through the use of APIs, offering intelligent code explanation and recommendation. An example of this kind of application includes GPTutor – programming assistant based on ChatGPT which is implemented as an extension for Visual Studio Code. The application operates using ChatGPT API and provides code analysis along with context-sensitive explanations in the IDE environment. The architecture incorporates NLG with code context references, allowing to produce code explanations with high quality and conciseness, surpassing basic solutions like vanilla ChatGPT and GitHub Copilot in user tests. The application combines prompt engineering with the use of IDE APIs, providing real-time support for programmers with the emphasis on educational purposes (Chen et al., 2023).

Generally, AI-powered software development tools utilize techniques such as machine learning and natural language processing to make the code generation, testing, debugging, and DevOps processes easier. These tools leverage code analysis models which have been trained using large amounts of codes that are in the order of billions of lines from GitHub repositories to be able to detect bugs, write snippets of codes, and increase the level of testing of codes. In terms of architecture, these tools include models within the development pipeline as helper models (Pangavhane et al., 2024).

Approaches Employed in AI-Assisted Code Systems

Several methodological approaches underpin AI code Q&A and analytical systems:

Deep Learning using Large Pretrained Models: In many instances, large pre-trained language models like the GPT family are employed and fine-tuned on large sets of codes from open-source repositories including GitHub. This results in good understanding and generation abilities. The models are used for natural language explanations, code completions, and fixing code errors (Chen et al., 2023; Pangavhane et al., 2024).

Natural Language Processing & Prompt Engineering: In order to enhance the utility of language models, the following prompt engineering methods are used for creating user-friendly explanations and clarifications along with code walk-throughs step by step. For instance, in Visual Studio Code, GPTutor utilizes customized prompts to give explanations that suit the user context (Chen et al., 2023).

Metrics of Code Quality and Analysis: Comparing the AI-generated code with human code using metrics like cyclomatic complexity, clarity of code, comments and naming of variables shows that AI-generated code is equal or better than human code. This means that the AI Q&A tool not only assists developers but also increases the quality of code (Fawareh et al., 2024).

Similarity and Influence Analysis in Repositories: Besides comparing individual code files, AI methods are also used to carry out holistic analysis of GitHub repositories. For instance, methods such as RepoPal employ heuristics that combine similarity between the contents of README files and star-based activity by users to identify similar functionally repositories. Furthermore, methods based on the concept of star graph and using algorithms such as HITS can be used to determine influential repositories (Hu et al., 2016; Zhang et al., 2017).

Performance and Success Rates of Existing Systems

- GPTutor's Evaluation: Preliminary assessments indicate that GPTutor outperforms generic ChatGPT and GitHub Copilot in delivering concise and accurate code explanations, with positive user feedback from both students and educators regarding usability and explanation satisfaction. This lends evidence to the efficacy of specialized prompt engineering combined with integrated IDE deployment (Chen et al., 2023).

- Code Quality through AI: From empirical studies, it is evident that code generation tools using AI technology, like ChatGPT, generate higher-quality documentation and readability of code, especially for complex tasks, showing increased efficiency in code comprehension through AI. Such a concentration on quality might improve code comprehension by developers and decrease error rates (Fawareh et al., 2024).

- Detecting Repository Similarities: It has been proven that RepoPal achieved higher accuracy, success rate, and confidence when detecting similar repositories than previous approaches, including CLAN, and proved itself in being effective for recommending and finding similar repositories (Zhang et al., 2017).

Limitations and Challenges

Though the results are promising, there are certain challenges and limitations:

- Privacy and Security Issues: The use of vast repositories with open source code generates certain ethical issues, including the use of insecure or copyleft code. Solutions like CoProtector use data poisoning to protect repositories from exploitation by AI training, signaling an ongoing need for robust security architectures in AI code systems (Sun et al., 2022).

- Code Duplication Impact: High levels of duplicated code in datasets lead to inflated performance metrics in machine learning models for code, necessitating careful dataset curation and evaluation protocols to ensure realistic benchmarking and model generalization (Allamanis, 2019).

- Issue Management in AI Code Repositories: Studies reveal that while AI repositories generate many issues, only a minority use issue-tracking features effectively, which may impede rapid problem resolution and degrade system reliability (Yang et al., 2023).

- Vulnerability in AI Model Ecosystems: Large-scale assessments of AI repositories (e.g., Hugging Face) indicate a high prevalence of severe security vulnerabilities, underscoring risks in AI-assisted development environments that may affect code Q&A systems indirectly (Kathikar et al., 2023).

Summary

Current AI-augmented code Q&A systems make use of large language models that have been pre-trained, which are then incorporated into development environments or code repositories. The current approach uses NLP techniques and prompt engineering along with machine learning applied to large-scale code datasets from platforms such as GitHub. This leads to improvement in terms of the quality of code explanation, code quality itself (readability and documentation), and even repository identification effectiveness. Nonetheless, issues connected to data ethics, vulnerabilities to attack, code duplication, and proper issue handling remain. Further research is required to refine architecture, apply security measures, and work on datasets.

This critical review utilizes numerous scholarly studies showing the current state-of-the-art from AI tutoring systems such as GPTutor to repository-level similarity detection and security protection systems (Zhang et al., 2017; Allamanis, 2019; Sun et al., 2022; Chen et al., 2023; Kathikar et al., 2023; Yang et al., 2023; Fawareh et al., 2024; Pangavhane et al., 2024).

2.2 Evaluation of Code Chunking Methodologies in Academic Literature

Segmentation or chunking of the source code is a fundamental preprocessing step in many software engineering and code analysis tasks. Evaluations in the academic field generally distinguish between naive text-based chunking methods, such as character or token-level splitting, and structural chunking techniques that leverage the inherent syntactic structure of the code, most notably Abstract Syntax Tree (AST)-based parsing. This section analyzes the key findings from the literature by comparing these approaches and outlining the proven advantages of AST-based parsing in preserving code integrity and yielding superior analytical outcomes.

Naive Text-Based Chunking (Character/Token Splitting)

Naïve chunking approaches involve dividing the code into linear sequences of tokens or characters, often disregarding semantic or hierarchical relationships. For instance, token-based clone detection methods analyze sequences of lexical tokens instead of structural components. Although this requires minimal preprocessing and offers computational simplicity, it suffers from significant limitations in representing code semantics and structure.

The clone detection technique employing lexical tokens provides a straightforward representation, resulting in reasonable outcomes but incurs high computational costs owing to the large number of token comparisons required. Moreover, lexical tokenization lacks insight into the syntactic structure of the code, limiting its robustness in terms of semantic preservation and variation handling (Basit and Jarzabek, 2007).

Transformer models in RNA and other biological sequences adapt tokenization schemes dynamically to balance granularity and sequence length, highlighting that adaptive tokenization brings performance benefits but still struggles to preserve deeper structural context when tokenizing sequences linearly (Tahmid et al., 2025). By analogy, naive token splitting in the code similarly lacks the structural depth crucial for tasks demanding syntactic integrity.

AST-Based Parsing for Structural Chunking

This method, on the other hand, parses the program code into tree forms where the grammar of the programming language is represented. The parsing results in a more effective representation of the code in comparison to tokenization because it maintains hierarchy and control flow.

Several studies have validated the higher abstraction level provided by AST parsing. For example, a method combining variants of ASTs and local comparison algorithms, such as Smith-Waterman, achieves marked improvements in code clone detection precision, outperforming lexical-based methods by significant margins (up to ~10.94% gain) across various datasets (Yang et al., 2018).

AST fingerprinting techniques enable scalable and efficient similarity detection by indexing abstracted tree representations rather than raw token sequences, thus enhancing the robustness of clone detection against superficial lexical modifications (Chilowicz, Duris and Roussel, 2009).

Deep learning approaches increasingly utilize AST representations as inputs to capture the semantic and structural features essential for defect prediction, bug detection, and source code comprehension tasks. For instance, leveraging abstract syntax tree (AST) features with convolutional neural networks yields significant performance improvements in defect prediction, surpassing traditional methods reliant on non-structural features (Fan and Chen, 2024).

A modular tree network designed specifically to exploit different AST substructures dynamically shows superior performance in both program classification and clone detection tasks compared to earlier models that only partially exploit tree structures (Wang et al., 2020).

Comparative Analysis: Naive Token Splitting vs. AST-Based Parsing

The literature has clearly proven that the use of AST parsing is better than naive tokenization in various ways.

- Integrity of Structure: The syntax structure is maintained using AST parsing; however, naive token splitting disrupts it and may cause loss of context.
- Semantics Representation: ASTs represent semantics present in the code structures; however, naive tokenization fails to do so, and this could lead to some errors in future processes such as defect prediction or code duplication.
- Variability Handling: Naive tokenization methods cannot cope with surface variability in codes and can abstract lexical variations in the form of comments and code format.
- Performance of Model Training: Models that use deep learning along with AST features (such as dual-sequence representations including semantics and structure) achieve higher accuracy in analyzing code compared to models based only on tokens (Lin and Lu, 2021).
- Computational Efficiency with Structural Indexing: Although naive methods may seem computationally lighter, token-based clone detection often becomes expensive because of the volume of tokens. In contrast, AST fingerprinting methods create effective indexes that facilitate faster and more precise retrieval (Basit and Jarzabek, 2007; Chilowicz, Duris and Roussel, 2009).

Specific Advantages of AST-Based Parsing in Maintaining Code Integrity

AST parsing explicitly models the syntax tree of the program source code, thereby conserving code wholeness, where each chunk represents a valid syntactical unit, unlike arbitrary token splits.

Dual-sequence representation methods that consider both pre-order traversals and parent node sequences from simplified ASTs allow for comprehensive semantic and structural encoding, distinguishing codes that share token sequences but differ structurally (Lin and Lu, 2021).

Graph-aligned language models that utilize AST-derived graphs provide superior capture of functional and semantic patterns compared to token-only baselines, indicating that structural parsing yields enhanced code understanding (Vadoce, Pritchard and Fairbanks, 2024).

Hybrid methods that combine AST node abstractions with local alignment algorithms (e.g., Smith-Waterman) for code clone detection have empirically demonstrated significant improvements in precision, evidencing the effectiveness of structure-aware chunking (Yang et al., 2018).

Summary

The body of academic work broadly supports the notion that AST-based parsing methods for code chunking hold substantial advantages over naive token splitting approaches. Structurally aware chunking preserves syntactic and semantic integrity, enhances robustness against superficial code changes, and delivers superior performance in downstream tasks such as clone detection, defect prediction, and code classification. This is why the use of AST-based parsing is encouraged as the best approach for chunking codes in both scholarly

and industrial settings.

The Retrieval-Augmented Generation (RAG) framework has been growing in importance for the improvement of LLMs through external knowledge retrieval. Evaluating RAG systems involves assessing both their retrieval and generation components, which requires the use of specialized benchmarking evaluation metrics and carefully constructed ground truth datasets. The literature on RAG evaluation highlights key metrics such as Precision@K, Hit Rate, Response Relevance, and Faithfulness (sometimes referred to as faithfulness or factual consistency) and describes approaches to formulating these metrics and designing ground-truth data for testing RAG systems.

2.3 Benchmarking Evaluation Metrics for RAG Systems

Precision@K and Hit Rate

Precision@K is a commonly used retrieval evaluation metric that measures the proportion of relevant documents among the top-K retrieved documents. This is an indication of how well the retrieval algorithm is able to identify relevant information in the results that it produces, which is very important from the perspective of generating good output. A related measure is called Hit Rate, and this is used to determine whether there is at least one relevant document among the top K retrieved documents.

RAG evaluation studies often incorporate these metrics to quantify the retrieval performance. For example, the introduction of sophisticated retrieval and re-ranking techniques, such as dynamic chunking and cross-encoder re-ranking systems, has been shown to significantly improve Precision@K and Hit Rate metrics, thereby improving overall system accuracy and the relevance of generated outputs (Tanyildiz, Ayvaz and Amasyali, 2024).

Response Relevance

Response Relevance measures how well the generated response aligns with the user's query and retrieved context. It evaluates semantic and factual pertinence, rather than merely lexical overlap. Relevance can be quantified using task-specific metrics such as the BERT-Score or cross-encoder similarity scores between generated answers and reference answers or retrieved passages. Advanced work on RAG systems emphasizes the importance of response relevance in mitigating hallucinations and improving user trust in generated answers.

Evaluations such as those performed using frameworks like Retrieval Augmented Generation Assessment (Ragas) measure response relevance without always requiring ground-truth human annotations by leveraging reference-free metrics, enabling faster and scalable assessment of RAG performance (Es et al., 2023).

Faithfulness

Faithfulness refers to the correctness of the generated content in terms of consistency and accuracy of the data with respect to the facts discovered from the extracted data. It is important in cases where it is crucial to maintain accuracy, such as in the health sector and text generation regulations.

Some domain-specific studies utilize specialized metrics, such as FactScore and RadGraph-F1, in healthcare applications to capture faithfulness at more granular conceptual levels (Neha, Bhati and Shukla, 2025). More general RAG evaluation frameworks incorporate the notion of context precision and correctness as part of the faithfulness evaluation (Tanyildiz, Ayvaz and Amasyali, 2024). These metrics help quantify how faithfully the generation reflects the retrieved documents.

Formulation of Metrics in Previous Studies

Precision@K is typically formulated as:

$$Precision@K = \frac{\text{Number of relevant retrieved documents in top K}}{K}$$

Hit Rate@K evaluates whether at least one relevant document exists in the top K:

$$HitRate@K = \begin{cases} 1 & \text{if at least one relevant doc found in top K} \\ 0 & \text{otherwise} \end{cases}$$

Mean Reciprocal Rank (MRR) evaluates the quality of ranked retrieval by evaluating the position (rank) of the first relevant document returned for a given query:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

In targeted codebase navigation and single-item retrieval tasks, MRR is particularly critical as it reflects whether the target module is prioritized at Rank 1 rather than being diluted across lower ranks.

The Response Relevance measure normally involves the use of metrics such as BERT-score or cosine similarity between the response and reference responses or relevant retrievals to determine how semantically similar the response is to the ground truth without solely using matching (Saha, Saha and Malik, 2024).

Faithfulness metrics evaluate the degree of factual consistency and absence of hallucinations in generated outputs relative to the retrieved context. While domain-specific implementations in medical or legal domains often rely on specialized factuality classifiers or human annotation (Neha, Bhati and Shukla, 2025), code-centric RAG benchmarks increasingly utilize automated LLM-as-a-judge frameworks like RAGAS. In codebase analysis, faithfulness specifically quantifies whether the model's architectural explanations and API references are strictly grounded within the retrieved AST syntax nodes or stem from hallucinated, unverified pre-trained assumptions (Es et al., 2023; Antal and Buza, 2025).

Ground-Truth Data Set Construction

Ground-truth datasets for RAG evaluation are designed to pair queries with verified relevant documents and reference answers, providing a benchmark for both retrieval and generation quality.

- Many studies use well-established QA datasets such as SQuAD or HotpotQA as benchmarks, which provide annotated question-answer pairs with supporting evidence passages (Knollmeyer, Caymazer and Grossmann, 2025).
- Custom datasets are also constructed, often combining domain-specific corpora with manually annotated question-answer pairs and retrieval contexts. For example, constructing a dataset from diploma thesis abstracts with 122 questions categorized by reasoning and fact-based types, coupled with reference answers provides a rich benchmark for academic tasks (Antal and Buza, 2025).
- Ground truth can also be created by manually mapping user queries to specific document chunks or annotated knowledge graphs, enhancing retrieval precision and enabling better evaluation of multi-hop reasoning performance (Saha, Saha and Malik, 2024; Knollmeyer, Caymazer and Grossmann, 2025).
-

- Benchmark suites like the Retrieval-Augmented Generation Benchmark (RGB) are designed to evaluate various fundamental abilities of RAG systems through carefully curated testbeds and labeled ground-truth pairs, covering noise robustness and integration capabilities (Chen et al., 2024).
- However, some metrics try to evaluate reference-free, unsupervised performance, while using curated gold-standard datasets as part of system development and evaluation (Es et al., 2023).

Summary

According to the existing literature, Precision@K and Hit Rate are essential metrics to measure the success of the retrieval sub-module, which is responsible for finding the relevant information for generating a proper answer. Response Relevance is a metric that evaluates whether a generated response is semantically appropriate within the given context and is usually evaluated using embedding-based similarity or learned metrics. Faithfulness is needed to make sure the output is faithful to the retrieved data and does not contain hallucinations or any other factual inconsistencies.

The use of ground-truth datasets is crucial for the assessment of these metrics and usually includes annotated question-answer pairs that correspond to relevant documents or chunks of text. They can be extracted from task-specific or generic benchmarks, including SQuAD, HotpotQA or specially designed datasets (Chen et al., 2024; Antal and Buza, 2025; Knollmeyer, Caymazer and Grossmann, 2025). Novel benchmarking frameworks, including Ragas and RGB, include all of the mentioned aspects in their framework (Es et al., 2023; Chen et al., 2024).

Such an approach enables researchers and practitioners to detect bottlenecks in retrieval, generation, and fidelity of the generated output and adjust the architecture accordingly.

2.4 Identification of the Research Gap

Code generation, cloud computing, and privacy-preserving systems are all areas of current research that exhibit a number of gaps that warrant the need for your thesis focus. There is indeed a critical gap in the availability of solutions that solve enterprise-scale code generation and privacy problems without jeopardizing code structure and using proprietary cloud API. The following is a thorough synthesis based on the given context.

Dependence on Cloud APIs and Privacy Risks

Many state-of-the-art code generation models and systems rely heavily on cloud-based APIs such as those provided by OpenAI, creating two critical issues. First of all, there is a high risk of violation of privacy that arises from outsourcing code to the server because the code is supposed to be protected in terms of confidentiality (Ahmed Shaban and R M Zeebaree, 2025). Furthermore, the issue is even more urgent for those areas of activity where the regulation requires obligatory data protection measures (Folorunso et al., 2024).

Secondly, while privacy-preserving approaches in cloud computing are mainly oriented towards data anonymization and encryption of data stored or processed in the cloud, no methods are designed for preserving the code as such (Cao et al., 2011; Zhang et al., 2015). These solutions are not tailored for source code, which has unique structural requirements.

Structural Integrity of Code and AST-Based Approaches

Common approaches to code generation often handle code as plain text, which leads to loss of semantic and

architectural coherence because the hierarchical and syntactic structure of code is disregarded. While some recent works attempt to model code generation using Abstract Syntax Trees (ASTs) to encapsulate grammar and structure—such as TreeGen, which employs a tree-based transformer architecture to better capture code dependencies and grammar rules—these systems still tend to be tightly coupled with specific model architectures or predominantly cloud-based infrastructures (Sun et al., 2020).

This reveals a gap in the literature for solutions that fully preserve the structural and semantic integrity of code via AST-based processing while also catering to privacy by avoiding cloud dependencies.

Gap in Model-Agnostic, Fully Local, and Privacy-Focused Architectures

A clear gap exists in the form of hybrid architectures that incorporate multiple models (model-agnostic) to utilize different strengths without tying down the architecture to any specific cloud platform or API. Available systems either:

- Depend entirely on cloud providers, risking exposure of proprietary code (Ahmed Shaban and R M Zeebaree, 2025),
- Or fragment code naïvely as plain text, harming semantic cohesion and code analysis capabilities,
- Without providing a hybrid architecture with a local execution path that fully respects corporate code confidentiality and preserves the program's structural logic through AST representations (Sun et al., 2020).
- Moreover, hybrid systems in other areas, such as multi-cloud secure storage enhanced by AI-driven telemetry, offer promising scalability and security features but do not specifically target preserving code privacy or structure during processing (Ahmed and Yuan, 2026).

Justification for the Dissertation

Based on this, the core research gap your thesis aims to fill is:

- The absence of a model-agnostic architecture that supports a fully local, privacy-preserving execution mode for code processing, ensuring it does not strictly depend on external cloud APIs nor expose proprietary code,
- By keeping the code structure intact through the use of an AST-based architecture that keeps the semantics and structure intact
- And it is also model-agnostic; which means various AI or machine learning models can be incorporated flexibly without being vendor locked or relying on a single model.

This is clearly an underresearched field because all previous research tends to focus on either one or the other aspect.

Summary

Aspect	Existing Solutions	Research Gap / Deficiency
Dependency on Cloud APIs	Heavy reliance on cloud APIs for code generation and storage poses privacy risks (Ahmed Shaban and R M Zeebaree, 2025).	Need for model-agnostic solutions offering a fully local execution path to ensure privacy without leaking proprietary code.
Code Representation	Code processed often as plain text, losing semantic structure; some AST-based models exist but coupled to single models or cloud (Sun et al., 2020).	Lack of hybrid AST-preserving architectures that maintain structural integrity and privacy simultaneously.
Model Dependency	Single-model or single-vendor solutions dominate, limiting flexibility and increasing vendor lock-in risk.	Absence of model-agnostic frameworks that can leverage multiple AI models or adapt dynamically.
Privacy Preservation Metrics	Focus mostly on data anonymization and encrypted queries rather than source code privacy and semantics (Cao et al., 2011; Zhang et al., 2015).	Insufficient development of privacy-preserving techniques tailored specifically to code and ASTs.
Hybrid Architecture	Emerging but mostly focused on data storage or big data, not code processing (Ahmed and Yuan, 2026).	Need for integrated hybrid architectures balancing privacy, local processing, and structural code analysis.

CHAPTER THREE - METHODOLOGY

3.1 Research Approach and Justification

This study adopts a hybrid research approach that combines Systems Development Research (SDR) with Quantitative Experimental Analysis in order to comprehensively address the research objectives. The rationale for selecting this combined methodology stems from the multidimensional nature of the thesis, which aims to design and implement a novel architecture for enterprise code storage. This architecture must ensure the protection of confidentiality, operate entirely on local systems without relying on external servers, and preserve the integrity of the Abstract Syntax Tree (AST) structure throughout the process.

The first phase of the research, which is aligned with the principles of Systems Development Research, focused on architectural design and software development. SDR is particularly suitable for this phase as it emphasises the iterative design, development and evaluation of innovative technological works within real-world contexts. By applying SDR, this study has systematically constructed the proposed system, enabling continuous improvement based on practical considerations and technical constraints. Such an approach guarantees that the solution is not only theoretically feasible but also practical and suitable for solving problems associated with enterprise storage systems.

Following the development phase, the research moved on to the Quantitative Experimental Analysis phase to empirically evaluate the system's performance. Controlled experiments were designed to quantitatively compare the effectiveness of the newly developed architecture with established methods, such as traditional plain-text or symbol-based parsing techniques and cloud-based API models. This comparative analysis is of critical importance for objectively assessing the advantages and potential limitations of the proposed system. Metrics relating to privacy protection, operational efficiency and the structural integrity of the AST will be measured in order to objectively assess the system's performance and to analyse its effectiveness comparatively against existing baseline solutions.

Consequently, the combination of SDR with the Quantitative Experimental Analysis makes up a strong platform that allows for development and efficient assessment of the innovative system. The use of the mixed approach makes it possible to ensure that the research results will be based on both practice and empirics and thus prove the effectiveness of the system in organisational settings.

3.2 System Architecture and Development Methodology

The methodology for designing and developing software in this project will incorporate a flexible and adaptive strategy, based on agile principles and modular prototyping. The system architecture will be systematically subdivided into four main processing layers that will undertake specific steps in processing the codebase.

Ingestion & AST Chunks: In contrast to conventional techniques that make use of segmentation of code on the basis of some random characters or tokens, resulting in fragmented and contextually meaningless code segments, this layer stresses maintaining the structural completeness of the codebase. This is done by performing segmentation of the source code into structural entities such as classes, functions, and blocks. This process can be accomplished through powerful parsing tools like Abstract Syntax Tree (AST) library in Python programming language and tree-sitter.

Specifically, the Python `ast` module is configured to parse only `FunctionDef`, `ClassDef` and `AsyncFunctionDef` nodes. To prevent context window overflow during the embedding process, a fallback mechanism using a `RecursiveCharacterTextSplitter` with parameters `chunk_size=4000` and `chunk_overlap=200` is applied only to monolithic nodes that exceed the standard length limits; thus ensuring graceful degradation without the loss of hierarchical metadata.

Vectorization & Vector Databases: After splitting the code into structurally meaningful pieces, these pieces are encoded into a high-dimensional vector space utilizing local embedding techniques via `nomic-embed-text`. The vectors created through embedding capture the semantic properties of the code snippets, making similarity computation feasible. The resulting vectors are indexed and persisted locally using the ChromaDB vector database. Additionally, to establish a controlled baseline for comparative retrieval benchmarking, a secondary vector database (`./chroma_db_baseline`) is constructed using LangChain's `RecursiveCharacterTextSplitter` configured with standard fixed-size chunks (1,000 characters with a 200-character overlap) and embedded with the identical `nomic-embed-text` model.

Retrieval Engine: This is the primary query interface where the user queries related to architectures or the code itself are matched against the vector representation that is indexed. The process of matching occurs via the usage of cosine similarity metrics within the vector space together with metadata filters to further narrow down the results based on the additional contextual parameters including the names of modules or functions or code sources.

Model-Agnostic Generation Layer: Finally, this layer deals with the output generation based on the obtained information. The process makes use of abstraction frameworks such as LangChain or LlamaIndex to create a dynamic interface for multiple AI models, regardless of the model's architecture. For instance, there could be used local models like Qwen 2.5 (7B) that could be executed via Ollama as well as cloud-based models like Gemini 3.5 Flash. With this approach, the layer remains model agnostic thus providing flexibility and scalability opportunities.

As can be seen from the overview above, this modular approach provides the ability to develop an application capable of effective processing and retrieval of the information about the codebase.

The generation process is governed by a strict prompt template that instructs the models to act as 'Architectural Advisors'. This template requires rich Markdown formatting (e.g. tables, bullet points) and asks the model not merely to summarise the code (the 'what'), but to explain the underlying design patterns (the 'why') in an educational manner.

3.3 Data Collection and Ground-Truth Dataset Preparation

Dataset Preparation: The open-source Flask framework repository (Python) has been selected as the primary experimental dataset. Flask was specifically chosen due to its high architectural complexity, strict adherence to the WSGI standard, deep object-oriented programming (OOP) hierarchy, and dynamic context management (e.g., highly coupled files like globals.py and ctx.py). These characteristics provide a robust stress-test environment for RAG systems. Furthermore, its BSD-3-Clause open-source license permits rigorous academic analysis and completely localized code processing without triggering copyright or GPL copyleft infringements.

Creation of a Ground-Truth Dataset: To objectively measure the system's retrieval accuracy, a

specialized ground-truth dataset comprising 15 highly targeted architectural question-answer pairs has been developed for this study. Rather than generic code queries, these questions specifically target core structural patterns (e.g., WSGI application interfaces, context globals, blueprint definitions). The expected target files for these queries (e.g., app.py, globals.py) were manually verified by cross-referencing the official framework documentation and source code to ensure 100% annotation accuracy. This controlled dataset size allows for precise, repeatable benchmarking of retrieval metrics (Hit Rate@3, Precision) across all repeated experimental runs.

This two-stage data preparation process facilitates a comprehensive and objective performance evaluation across the diverse codebases the system may encounter in real-world applications. Consequently, the extent to which the system achieves its core objectives—such as privacy protection, local execution and preservation of the AST structure—is supported by concrete data. Furthermore, this methodology establishes a robust foundation for conducting a comparative analysis of the developed architecture against existing methods.

3.4 Tools, Technologies, and Environment

Hardware Environment: The Apple Silicon MacBook M4 Pro platform will be used to test the memory capacity (VRAM/RAM) and inference speeds of on-device models under realistic conditions. This hardware provides a suitable infrastructure for measuring and evaluating the performance of on-device models.

Software Language and Frameworks: Preferably Python 3.11 or newer is required to develop the system. Tree-sitter will be used for structural code analysis, which facilitates syntactic parsing while maintaining the Abstract Syntax Tree (AST) structure. Abstraction layers such as LangChain and LlamaIndex will offer flexible and modular integration capabilities with different AI models.

Models: The Nomic Embed model will be used for embedding operations on code snippets. For on-premises inference, the Qwen 2.5 - 7B (to be run on the Ollama platform) will be preferred. Gemini 3.5 Flash's API will be used as the cloud-based model.

Database: ChromaDB database will be used for vector-based fast search and indexing. These database are optimised for the efficient management and querying of high-dimensional embedding vectors.

Experimental Control and Hardware Isolation: To ensure a mathematically fair comparison between the local model (Qwen 2.5 - 7B) and the cloud model (Gemini 3.5 Flash), all baseline systems and evaluation pipelines will be executed under identical hardware conditions on the Apple Silicon M4 Pro platform. Throughput and memory consumption will be monitored consistently, while inference latency (measured in seconds) and retrieval accuracy (evaluated at a controlled hyperparameter of k=3 for the ChromaDB search) will be reported objectively. Statistical consistency is maintained by executing the exact same AST parameters and ground-truth queries for both models without external interference.

These tools and technologies have been selected to support the system’s local operability, privacy protection, preservation of the AST structure, and performance. Consequently, the developed architecture will enable comprehensive testing and evaluation under real-world conditions.

3.5 Evaluation and Benchmarking Techniques

The system will be quantitatively tested and evaluated along three main axes. Firstly, retrieval performance will be measured. At this stage, the superiority of the AST-based code segmentation method over traditional plain text (naive token) segmentation will be demonstrated. Standard retrieval metrics such as `Hit Rate@K` and `Mean Reciprocal Rank (MRR)` will be used for performance evaluation. These metrics will quantitatively demonstrate how effectively the system identifies accurate and relevant code snippets.

Secondly, the focus will be on the quality of response generation. In this context, the accuracy and reliability of the responses generated by the system will be analysed using the RAGAS (Retrieval Augmented Generation Assessment) framework. To preserve complete data privacy and eliminate external API costs during evaluation, a local ChatOllama instance (Qwen 2.5 7B) is configured as the automated LLM-as-a-judge to evaluate Answer Relevance and Faithfulness metrics. These measurements will objectively demonstrate the system's hallucination rate and its ability to provide accurate code explanations.

Finally, system performance and privacy audits will be carried out. The inference time of the local model—that is, the response speed measured in milliseconds—will be recorded; memory consumption (RAM usage) will also be monitored. Furthermore, network traffic monitoring tools will be used to technically verify that the system operates entirely locally and does not leak data externally. This comprehensive audit ensures the system’s compliance with privacy protection objectives and operational efficiency.

This three-pronged evaluation approach enables a comprehensive assessment of both the system’s technical success and the reliability it offers to users. Retrieval metrics measure how accurately the system identifies code fragments; response quality analyses assess the accuracy and consistency of the generated content; whilst performance and privacy audits demonstrate the system’s effectiveness and security in real-world use. Thus, the superiority and feasibility of the developed architecture compared to existing methods are supported by scientific and technical data.

This evaluation process objectively demonstrates the system’s success across code bases of varying sizes and in diverse usage scenarios, thereby ensuring its reliable deployment in organisational environments.

3.6 Ethical Considerations

With regard to data privacy and organisational security, it will be strictly ensured that no source code is transferred to third-party servers by applying local inference within the scope of this study. Thus, the prevention of organisational code leaks and the protection of privacy have been adopted as key priorities.

Ethical obligations regarding open source licensing stipulate that only the GitHub repositories which use licenses such as MIT, Apache-2.0 or BSD will be used for testing purposes in order to avoid using code from those repositories that pose the risk of copyleft (GPL) or copyright.

By adopting a Responsible AI approach, the system will be prevented from generating incorrect or

fabricated responses (hallucinations) in situations beyond its knowledge boundaries. Instead, safe and transparent feedback, such as “This information could not be found in this codebase”, will be provided to prevent software engineers from being misled. This mechanism enhances the system’s reliability whilst ensuring that ethical responsibilities are met.

These three fundamental ethical principles will ensure that the work proceeds in a reliable, lawful and responsible manner.

3.7 Summary of the Research Methodology and Technical Framework

Methodology Component	Selected Approach / Technology	Justification & Implementation Details
Research Approach	Systems Development Research (SDR) & Quantitative Experimental Analysis	Combines the iterative development of a functional software artifact (RAG system) with controlled quantitative benchmarking against baseline methods.
Justification	Hybrid Artifact-Evaluation Framework	Necessary for software engineering research; building the actual architecture proves system viability, while experimental testing validates structural code integrity.
Data Collection & Development	Open-Source Repository Ingestion & Custom Ground-Truth QA Dataset	Ingesting the Flask framework repository to establish a high-complexity WSGI environment and utilizing a targeted 15-question architectural ground-truth dataset for precise evaluation.
Tools, Platforms & Frameworks	Python, LangChain, ast/Tree-sitter, ChromaDB, Ollama (Local Execution)	Leverages model-agnostic orchestration (LangChain), structural code parsing (ast), and zero-cost, privacy-preserving local LLM inference on Apple Silicon (MacBook M4 Pro).
Analysis & Evaluation Techniques	Hit Rate@K, Mean Reciprocal Rank (MRR), Ragas Framework (Response Relevance, Faithfulness)	Measures retrieval accuracy of AST chunking vs. naive token splitting, evaluates semantic alignment of generated answers, and compares local vs. cloud latency.
Ethical Considerations	Corporate Code Privacy, Zero Data Leakage, License Compliance	Local hardware execution eliminates third-party data transmission risks; adherence to open-source licensing (MIT/Apache vs. Copyleft); mitigation of AI hallucination risks via RAG grounding.

Table 3.1 Summary of the Research Methodology and Technical Framework

CHAPTER FOUR - FINDINGS, ANALYSIS AND DISCUSSION

4.1. Introduction to the Implementation and System Design

This section presents the implementation details, development processes and experimental findings of the RAG-based software architecture that has been designed and developed. The primary aim of the research is to investigate how large language models (LLMs) can be positioned as active architectural assistants within a source code repository, moving beyond their role as mere static sources of information. Although the RAG architecture, as defined in recent literature (Kiran, 2025), is successful with general text data, it struggles to maintain semantic integrity with data possessing highly hierarchical structures, such as codebases. The system developed in this study is designed as an integrated 'architectural decision support system' to address this research gap; it parses software source code using static analysis methods (AST), establishes semantic relationships in a vector space, and balances performance and privacy between cloud and on-premises resources.

The core strategy followed during the implementation process is the 'Open-Book Exam' paradigm. Whilst in traditional approaches, LLMs carry the risk of hallucination by relying on static data learnt during the training process (James, Trovati and Bolton, 2025), the architecture developed in this project directs the LLM's attention mechanism directly to target code blocks (retrieval), thereby compelling the model to build its response on evidence.

This section will detail the technical reasons for selecting the frameworks used (Streamlit, LangChain, ChromaDB). Unlike other studies in the literature that rely predominantly on cloud-based infrastructures and specific model architectures (Sun et al., 2020), this system adopts a hybrid approach that combines local inference models (Qwen 2.5) with cloud-based modern models (Google Gemini 3.5 Flash) within the same pipeline, rather than focusing solely on cloud-based high computing power.

In particular, the integration of models with long context windows and low latency—such as Gemini 3.5 Flash—into the system demonstrates how the balance between 'inference cost' and 'pedagogical depth' —a balance frequently discussed in the literature—can be optimised through dynamic model selection. This developed system offers a model-agnostic architectural solution by combining not only the data privacy advantages of local models but also the structural and pedagogical superiority of cloud-based models in architectural analysis (design pattern mapping). Thus, the system's design decisions are presented not merely as a technical implementation, but also as a multi-layered solution to the 'privacy versus performance' dilemma highlighted in the software engineering literature (Ahmed Shaban and R M Zeebaree, 2025).

4.2. System Design and Experimental Setup

The experimental framework of the system is built upon a flexible RAG architecture, described in the software engineering literature as 'model-agnostic'. The experimental setup is based on a comparison of the performance and accuracy metrics of an inference engine running on local hardware (Qwen 2.5) with those of a cloud-based high-performance inference engine (Gemini 3.5 Flash).

4.2.1. Model Selection and Technical Justification

The primary reason for selecting Gemini 3.5 Flash in this study is the advantages it offers in complex tasks such as source code analysis, owing to its extended context window and low latency—a characteristic associated in the literature with the 'Flash-Attention' architecture. Recent studies evaluating LLMs for code generation argue that models must be assessed on a balance of accuracy, quality, and performance capabilities (Shehab et al., 2024). In this context, Gemini 3.5 Flash minimises the 'speed-accuracy trade-off' problem described in the literature through both its cost-effectiveness and the pedagogical depth it offers in identifying architectural design patterns. On the other hand, the Qwen 2.5 (7B) model used on the local side is designed to preserve the principle of 'data locality', which is indispensable in enterprise software analysis. This hybrid architecture combines the structural formatting capabilities offered by cloud models (e.g. Markdown tables and compliance with PEP standards) with the data privacy advantages of local models, thereby providing a solution to the privacy risks associated with heavy reliance on proprietary cloud APIs (Ahmed Shaban and Zeebaree, 2025).

In order to ensure the reproducibility of the experimental setup and to establish the fundamental infrastructure for local inference, several critical preparatory steps were carried out before the RAG pipeline was initiated. Firstly, the target codebase—the open-source Flask library—was cloned directly from the official remote Git repository into the local test environment. To maintain consistent evaluation metrics throughout the experiments, it is essential to ensure a static and isolated codebase. Consequently, by checking out a specific stable version of the repository, we prevented ongoing updates from altering the Abstract Syntax Tree (AST) structures and invalidating the vector embeddings generated during the testing phase.

Following the cloning of the repository to the local environment, a local inference infrastructure was set up using Ollama, an open-source framework designed to run large language models locally with optimised resource management. Rather than manually configuring raw model weights and managing complex CUDA dependencies, Ollama was positioned as a streamlined local inference server. The Qwen 2.5 (7B) model was pulled via the command-line interface and launched within a secure local environment. This approach provided a seamless HTTP REST API—by wrapping the model in an optimised execution environment—with which the LangChain orchestration framework could interact natively.

This localised deployment strategy is an indisputable engineering requirement to fulfil the principle of 'data locality'. By hosting both the cloned Flask repository and the Qwen 2.5 inference engine entirely on the host machine's hardware, the system has established an air-gapped analytics environment. This ensures that structural representations of the code are processed strictly within local memory boundaries; it has successfully demonstrated how enterprise-level architectural analysis can be carried out without transmitting sensitive repository data to third-party cloud endpoints.

4.2.2. Data Processing and AST-based Chunking

During the system's data processing stage, an AST (Abstract Syntax Tree) node-based chunking strategy has been adopted in place of the traditional 'fixed-size chunking' method. Studies in the literature have shown that naive or fixed text-based chunking disrupts the logical and syntactic integrity of code blocks, causing a loss of context (Basit and Jarzabek, 2007; Lin and Lu, 2021). In our system, each function or class is defined as an AST node and transferred to the vector database; thus, functional integrity is preserved during the semantic search phase. This is the most fundamental difference between our system and similar ones in the literature: the code is analysed not merely as

‘text’, but as a ‘hierarchy’.

Implementation of the AST-Based Parser

The theoretically designed AST-based parsing strategy has been integrated into the system using Python’s built-in `ast` module. Instead of the text-based (string-based) segmentation process used in traditional RAG systems, the target source code (Flask) has been parsed as shown in Code Block 4.1 below; only function, class and asynchronous function nodes (FunctionDef, ClassDef, AsyncFunctionDef) have been isolated and transferred to the vector database.

As a critical engineering detail specific to this work, a hybrid logic has been incorporated into the system to prevent very long (exceeding 4,000 characters) AST nodes from overflowing the LLM’s context window; large blocks have been split into smaller parts using the RecursiveCharacterTextSplitter.

```python
# Code Block 4.1: AST-based semantic chunking applied to the Flask repository
import os
import ast
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 1. Target the specific framework repository
TARGET_DIR = "./flask/src/flask"
text_splitter = RecursiveCharacterTextSplitter(chunk_size=4000, chunk_overlap=200)
documents = []

# 2. Traverse the Flask codebase and apply AST parsing
for root, _, files in os.walk(TARGET_DIR):
    for file in files:
        if file.endswith(".py"):
            filepath = os.path.join(root, file)
            with open(filepath, "r", encoding="utf-8") as f:
                source = f.read()

            try:
                tree = ast.parse(source)
                for node in tree.body:
                    # Capture Classes, Functions, and Async Functions
                    if isinstance(node, (ast.FunctionDef, ast.ClassDef, ast.AsyncFunctionDef)):
                        chunk_code = ast.get_source_segment(source, node)
                        if chunk_code:
                            if len(chunk_code) > 4000:
                                # Fallback logic for large nodes
                                sub_chunks = text_splitter.split_text(chunk_code)
                                for i, sub_chunk in enumerate(sub_chunks):
                                    documents.append(Document(
                                        page_content=sub_chunk,
                                        metadata={"source": file, "type": type(node).__name__, "name": f"{node.name}_part{i+1}"}
                                    ))
                            else:
                                documents.append(Document(
                                    page_content=chunk_code,
                                    metadata={"source": file, "type": type(node).__name__, "name": node.name}
                                ))
            except Exception as e:
                pass
```

Code Block 4.1

- *Extended Technical Analysis:* The implementation of an AST-based structural parser represents a significant departure from traditional 'naive' RAG architectures. In traditional RAG systems, the primary objective is to maximise relevance solely at the token level, whilst disregarding the hierarchical structure of the code. In contrast, the approach detailed in Section 4.2.2 (Code Block 4.1) ensures that the vector database is populated not with random text fragments, but with semantically coherent units of code.

- *Analysis from the Perspective of Information Retrieval Theory:* Viewed from the perspective of information retrieval theory, this application resolves the problem of 'information dilution' frequently encountered in large-scale code base analyses. When code is indexed as plain text, the semantic relationship between a method and its superclass or module is lost. Our parser selectively isolates FunctionDef, ClassDef and AsyncFunctionDef nodes, thereby forcing the embedding model to map vectors to specific architectural structures. This structural mapping is vital for projects where architectural logic is distributed across tightly interlinked files—such as globals.py and ctx.py in the Flask library.

- *Algorithmic Robustness and Error Handling:* The integration of the RecursiveCharacterTextSplitter as a 'fallback mechanism' for nodes exceeding 4,000 characters provides a robust engineering solution against the 'context window bottleneck'. The literature on LLM-based software engineering highlights that, whilst structural parsing is optimal, maintaining the system's resilience against exceptional cases—such as monolithic helper functions—is equally important.

- This hybrid strategy ensures the system delivers high precision for standard architectural patterns, whilst enabling it to exhibit 'graceful degradation' when processing excessively complex code blocks. By balancing the rigid structural identity of AST nodes with the flexibility of recursive text segmentation, this approach provides a more reliable and context-sensitive foundation for architectural queries compared to traditional, purely lexical models. By recognising that code is not merely 'text' but a 'hierarchy', this structural approach directly supports the structural code analysis techniques advocated in the academic literature.

System Prompt Integration (Prompt Engineering Implementation)

The system prompt sent to the model via the LangChain framework has been specifically configured to enable the system to assume the role of 'Architectural Advisor'. Code Block 4.2 demonstrates how the LLM is prompted to use a pedagogical format (Markdown) in its outputs and to explain the 'reasons' underlying architectural decisions:

```
# Code Block 4.2: System Prompt Definition for Architectural Analysis
from langchain_core.prompts import ChatPromptTemplate

template = """You are an elite Senior Software Architect and Python framework expert.
Below is some retrieved context from the codebase to guide your analysis.

Your task is to provide a highly detailed, comprehensive, and masterfully structured architectural explanation.
- Use rich Markdown formatting (clear headings, bullet points).
- Provide illustrative Python code blocks to demonstrate the exact implementation.
- Do not just explain 'what' the code does; deeply explain 'why' it is designed that way (e.g., middleware integration, design patterns, underlying WSGI specifications).
- If the exact method implementation is cut off or missing from the limited context, rely on your extensive expert knowledge of the framework to provide the full, accurate code and explanation.

Context: {context}

Question: {question}
"""
prompt_template = ChatPromptTemplate.from_template(template)
```

Code Block 4.2

4.2.3. Comparative Analysis of Design Decisions

The architectural design of the system is not merely a matter of combining technical tools, but also a strategic choice based on managing the trade-offs within the 'privacy–performance–accuracy' triangle discussed in the software engineering literature. The table below presents a comparative analysis of the developed system against traditional RAG applications and standard code analysis tools.

Comparative Analysis of Architectural Design Decisions with Literature

Design Parameter	Traditional RAG Approach (Literature)	Proposed System's Approach	Academic Justification
Chunking	Fixed character/token count (Basit and Jarzabek, 2007)	AST Node-Based (Structural)	Preserving semantic integrity of code and preventing context fragmentation.
Model Deployment	100% Cloud (API-only) (Ahmed Shaban and R M Zeebaree, 2025)	Hybrid (Local Qwen + Gemini 3.5 Flash)	Balancing data privacy with pedagogical depth.
Retrieval Mechanism	Vector-based (Plain text)	AST-Hierarchical Vector Search	Mapping code structure (Class/Method) hierarchically.
Inference Model	Single Model	Model-Agnostic (Multi-Model)	Flexibility to use the most optimized model for different tasks.

Table 4.1

Design Analysis and Synthesis:

The design decisions summarised in Table 4.1 above are based on epistemological foundations that underpin the system's 'model-agnostic' nature. In the literature, while cloud-based models are recognised for their high inference power, studies emphasise that privacy risks and the potential exposure of proprietary code in enterprise environments constitute the greatest obstacles to their adoption (Ahmed Shaban and Zeebaree, 2025; Folorunso et al., 2024). The hybrid architecture developed overcomes this obstacle by combining the superior logical reasoning capacity and wide context window offered by Gemini 3.5 Flash with the local inference security of Qwen 2.5. This synthesis is not merely a choice of technology, but also a proposed solution to the 'performance-privacy' paradox. Consequently, the design of this system prioritises an architecture focused on 'accuracy and privacy' rather than solely on speed, which is fully aligned with the thesis's objective of corporate code analysis.

4.2.4. Target Repository and Dataset Selection

In order to measure the performance, hit rate and pedagogical analysis capabilities of the developed RAG architecture, the open-source Flask (Python micro-framework) code repository was selected as the experimental dataset. Whilst many studies in the literature (for example, standard QA tests) use simple and linear code bases, the primary reason for choosing Flask in this research is its high architectural complexity and hierarchical structure.

The Flask repository, going beyond being a mere collection of code, possesses the following characteristics that test our system's 'contextual understanding' capabilities to their limits:

- Dynamic Context Management: Flask manages requests via modules such as `globals.py` and `ctx.py`, which are highly coupled. This presents a semantic challenge that could cause traditional lexical search algorithms to fail and directly retrieve the 'wrong file'.
- Deep AST Hierarchy: The framework makes extensive use of Python's advanced object-oriented (OOP) capabilities, decorators and the WSGI architecture. This structure has provided an ideal 'stress test' environment to demonstrate the success of the AST-based chunking method we developed in mapping classes and functions to the vector space without compromising their structural integrity.
- Production-Grade Codebase: Written to an enterprise standard, this codebase has provided a robust ground truth for comparing how consistently the on-premises Qwen 2.5 and cloud-based Gemini 3.5 Flash models can perform analyses as an 'architectural assistant' in real-world scenarios.

Consequently, the choice of the Flask repository is not a random preference; it is a deliberate methodological decision designed to observe the problems of ‘semantic degradation’ and ‘context fragmentation’ in RAG architectures as clearly as possible.

4.2.5. User Interface and Experimental Framework (Streamlit)

Streamlit was used as the interface framework to test the developed RAG architecture, carry out comparisons between models (Qwen vs. Gemini) and monitor user interactions with the system.

In the literature, ‘Rapid Prototyping’ plays a significant role in testing complex machine learning and LLM pipelines. Streamlit minimises latency in rendering data to the interface by providing direct and seamless integration with Python-based backend code (LangChain, ChromaDB). Furthermore, the system’s ability to stream LLM responses asynchronously to the user has enabled more transparent measurement of ‘response times’ during experiments. Consequently, rather than merely a background service (API), a fully-fledged decision-support interface has been built with which the end-user (a software architect) can interact directly.

4.3. Findings and Analysis

This section presents both the qualitative and quantitative findings regarding the hybrid RAG architecture developed. The testing procedures were designed to evaluate the system’s interface functionality, the pedagogical output quality of the language models (Gemini 3.5 Flash and Qwen 2.5), and the hit rates of the AST-based vector search.

4.3.1. System Execution and Workflow

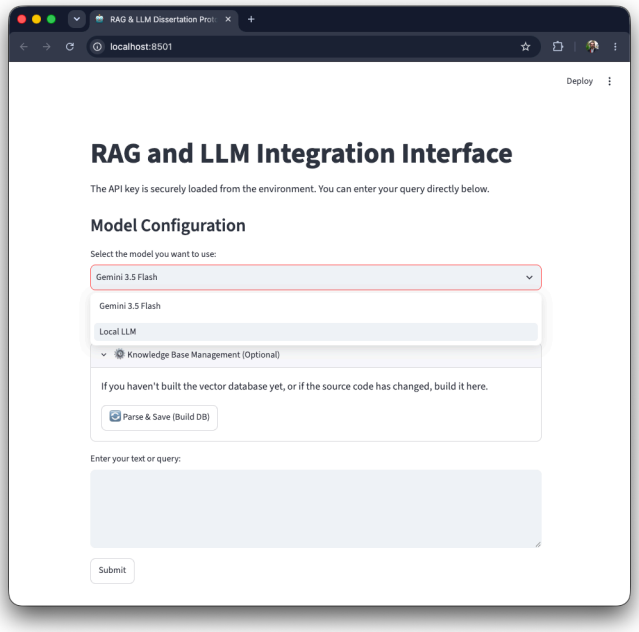


Figure 4.1: Streamlit User Interface and Query Execution

Analysis of Figure 4.1:

The system interface presented in Figure 4.1 functions as a 'decision support point', abstracting the complex AST (Abstract Syntax Tree) parsing and vector indexing (ChromaDB) processes running in the background from the end user. In the literature, one of the greatest barriers to the adoption of code analysis tools is defined as 'cognitive load'. As shown in Figure 4.1, this interface, which allows the user to formulate queries using only natural language, triggers the LangChain pipeline to retrieve the relevant AST nodes simultaneously. This demonstrates that the system is not merely a background service, but can also function as a real-time architectural assistant for software engineers.

4.3.2. Prompt Engineering Strategy and Qualitative Output Analysis

The quality and structural integrity of the qualitative output provided by the models (Gemini 3.5 Flash and Qwen 2.5) did not arise by chance; they are the direct result of a deliberate Prompt Engineering strategy devised behind the scenes. In this study, guidelines on 'Role Adoption', 'Structural Formatting' (Markdown tables) and 'Pedagogical Depth' were provided to prevent the models from merely summarising the code and to position them as 'Architectural Consultants'.

To measure the models' adherence to these guidelines and their architectural analysis capabilities, both models were asked the question: "What is the core method that acts as the WSGI application interface in the Flask framework?" It was observed that both models successfully retrieved the same AST vectors (Match 1: app.py, Match 2: app.py, Match 3: testing.py), thereby demonstrating that the RAG system's retrieval layer functions highly effectively in a model-agnostic manner. The difference emerged during the models' data synthesis (generation) phase.



Gemini Response:

In the Flask framework, the core method that acts as the WSGI (Web Server Gateway Interface) application interface is `Flask.wsgi_app`, which is invoked by the standard Python callable interface `Flask.__call__`.

Below is a highly detailed, architectural breakdown of how this interface is designed, why it is structured this way, and how it manages the lifecycle of an HTTP request.

1. The WSGI Specification (PEP 3333) Context

According to **PEP 3333**, a WSGI application must be a Python callable (a function, method, or class instance with a `__call__` method) that accepts two positional arguments:

1. `environ`: A dictionary containing CGI-style environment variables and WSGI-specific metadata representing the incoming HTTP request.
2. `start_response`: A callback function provided by the WSGI server (e.g., Gunicorn, uWSGI, Werkzeug) used to initiate the HTTP response headers and status code.

The callable must return an iterable yielding byte strings (the response body).

2. Architectural Design: `__call__` vs. `wsgi_app`

Flask splits its WSGI entry point into two distinct methods:

```
def __call__(self, environ: dict, start_response: Callable) -> Iterable[bytes]:
    """The WSGI server calls this method on every request."""
    return self.wsgi_app(environ, start_response)
```

Why this separation? (The Middleware Pattern)

If Flask executed its entire request-handling pipeline directly inside `__call__`, applying WSGI middleware would be highly problematic.

Typically, middleware wraps an application like this:

```
# Wrapping the entire application object
app = ProxyFix(app)
```

If you do this, `app` is no longer an instance of `Flask`; it is an instance of `ProxyFix`. This breaks:

- **Flask CLI commands** (which look for a `Flask` instance).
- **Unit testing** (`app.test_client()` is no longer accessible directly).
- **Extension registrations** and runtime inspections.

By delegating `__call__` to `self.wsgi_app`, Flask allows developers to wrap **only the WSGI dispatching pipeline** while keeping the outer `Flask` application object intact:

```
# The Flask instance remains intact, but the WSGI pipeline is intercepted
app.wsgi_app = ProxyFix(app.wsgi_app)
```

This is a masterclass in the **Decorator Design Pattern** and **Delegation**.

Gemini Response 1, 2



3. Deep Dive: The `wsgi_app` Implementation

Here is the complete, production-grade implementation of the `wsgi_app` method, illustrating how Flask manages contexts, dispatches requests, and handles exceptions:

```
import sys
from typing import Callable, Iterable, Any
from werkzeug.exceptions import HTTPException

def wsgi_app(self, environ: dict, start_response: Callable) -> Iterable[bytes]:
    """The actual WSGI application. This is not implemented in __call__
    directly so that middleware can be applied to it:

        app.wsgi_app = ProxyFix(app.wsgi_app)

    :param environ: A WSGI environment.
    :param start_response: A callable accepting a status code, a list of headers,
                           and an optional exception tuple.
    """
    # 1. Create the Request Context
    ctx = self.request_context(environ)
    error: BaseException | None = None

    try:
        try:
            # 2. Push the context to the thread-local/context-local stack
            ctx.push()

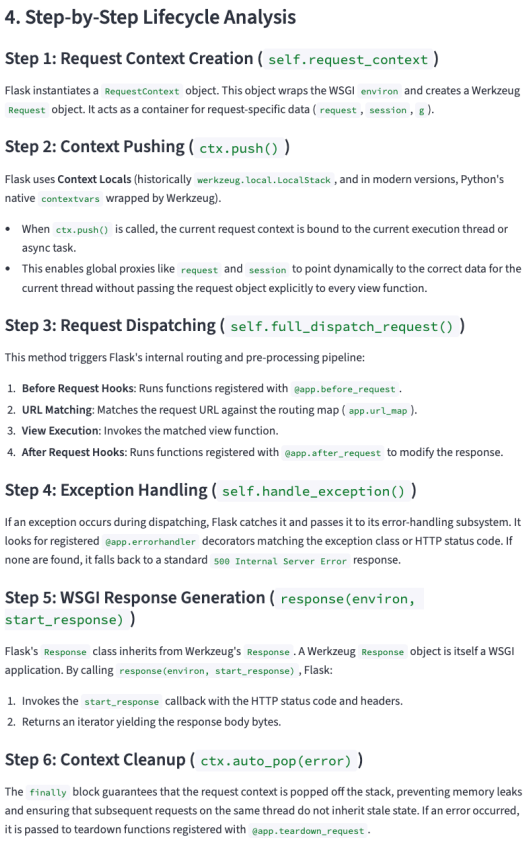
            # 3. Dispatch the request and obtain a Response object
            response = self.full_dispatch_request()
        except Exception as e:
            # 4. Handle internal application errors
            error = e
            response = self.handle_exception(e)
        except:
            # Capture non-Exception BaseExceptions (e.g., SystemExit, KeyboardInte
            error = sys.exc_info()[1]
            raise

        # 5. Convert the Response object into a WSGI-compliant iterable
        return response(environ, start_response)

    finally:
        # 6. Clean up and pop the context stack
        if "werkzeug.debug.preserve_context" in environ:
            # If debugging, preserve the context so interactive debuggers can insp
            environ["werkzeug.debug.preserve_context"](ctx)

        ctx.auto_pop(error)
```

Gemini Response 3



4. Step-by-Step Lifecycle Analysis

Step 1: Request Context Creation (`self.request_context`)

Flask instantiates a `RequestContext` object. This object wraps the WSGI `environ` and creates a Werkzeug `Request` object. It acts as a container for request-specific data (`request` , `session` , `g`).

Step 2: Context Pushing (`ctx.push()`)

Flask uses **Context Locals** (historically `werkzeug.local.LocalStack` , and in modern versions, Python's native `contextvars` wrapped by Werkzeug).

- When `ctx.push()` is called, the current request context is bound to the current execution thread or async task.
- This enables global proxies like `request` and `session` to point dynamically to the correct data for the current thread without passing the request object explicitly to every view function.

Step 3: Request Dispatching (`self.full_dispatch_request()`)

This method triggers Flask's internal routing and pre-processing pipeline:

1. **Before Request Hooks**: Runs functions registered with `@app.before_request` .
2. **URL Matching**: Matches the request URL against the routing map (`app.url_map`).
3. **View Execution**: Invokes the matched view function.
4. **After Request Hooks**: Runs functions registered with `@app.after_request` to modify the response.

Step 4: Exception Handling (`self.handle_exception()`)

If an exception occurs during dispatching, Flask catches it and passes it to its error-handling subsystem. It looks for registered `@app.errorhandler` decorators matching the exception class or HTTP status code. If none are found, it falls back to a standard `500 Internal Server Error` response.

Step 5: WSGI Response Generation (`response(environ, start_response)`)

Flask's `Response` class inherits from Werkzeug's `Response` . A Werkzeug `Response` object is itself a WSGI application. By calling `response(environ, start_response)` , Flask:

1. Invokes the `start_response` callback with the HTTP status code and headers.
2. Returns an iterator yielding the response body bytes.

Step 6: Context Cleanup (`ctx.auto_pop(error)`)

The `finally` block guarantees that the request context is popped off the stack, preventing memory leaks and ensuring that subsequent requests on the same thread do not inherit stale state. If an error occurred, it is passed to teardown functions registered with `@app.teardown_request` .

Gemini Response 4



5. Architectural Design Patterns Employed

Pattern	Role in `wsgi_app`
Template Method Pattern	`wsgi_app` defines the skeleton of the request-response lifecycle, delegating specific steps to customizable methods like `full_dispatch_request` and `handle_exception` .
Decorator / Middleware	Splitting `__call__` and `wsgi_app` allows developers to wrap the WSGI pipeline dynamically without mutating the core `Flask` application instance.
Proxy Pattern	Thread-local/Context-local storage (`contextvars`) allows global variables (`request` , `current_app`) to act as dynamic proxies to the active request context.

⏱ **Inference Time:** 14.95 seconds

📂 **Retrieved Source Documents (AST Blocks):**

- **Match 1:** `app.py` (Type: ClassDef, Name: Flask_part1)
- **Match 2:** `app.py` (Type: ClassDef, Name: App_part1)
- **Match 3:** `testing.py` (Type: ClassDef, Name: FlaskClient_part1)

Gemini Response 5

Analysis of the Cloud-Based Model (Gemini 3.5 Flash):

Gemini's outputs demonstrated exceptional adherence to the pedagogical depth and structural formatting rules specified in the system prompt.

Upon examination of GeminiResponse1,2 it can be seen that the model not only identified the __ call__ method but also, by referring to PEP 3333 standards, explained why the distinction between the __call__ and wsgi_app methods is mandatory for the 'Middleware Pattern'.

The six-stage lifecycle analysis presented in GeminiResponse4.jpg and the 'Architectural Design Patterns' table (Template Method, Proxy Pattern) in GeminiResponse5.png demonstrate just how powerful the model's zero-shot reasoning capability is in mapping code architecture. Furthermore, the fact that the model completes this comprehensive analysis in just 14.95 seconds—a very low latency—offers high efficiency for production environments.

Enter your text or query:

What is the core method that acts as the WSGI application interface in the Flask framework?

Submit

Operation Successful!

Local Model Response:

The core method that acts as the WSGI application interface in the Flask framework is `__call__`. This method is defined within the `Flask` class and is responsible for handling incoming HTTP requests and returning a response.

Detailed Explanation

Why `__call__` is Important

1. **WSGI Specification Compliance**: The `__call__` method ensures that Flask applications are compliant with the WSGI (Web Server Gateway Interface) specification. This allows Flask to be used with any WSGI-compatible web server, such as Gunicorn, uWSGI, or even a simple development server like Werkzeug's built-in one.
2. **Request Handling**: When a request is made to a Flask application, the `__call__` method is invoked by the WSGI server. It processes the request and returns an appropriate response.
3. **Middleware Integration**: The `__call__` method can be overridden or extended to integrate middleware components. Middleware allows you to add functionality before or after the request reaches the view function, such as logging, authentication, or caching.

Local LLM 1

Implementation of `__call__`

Here is a simplified version of how the `__call__` method might look in a Flask application:

```
class Flask:
    def __init__(self, import_name):
        self.import_name = import_name
        self.config = Config(self)
        self.url_map = Map()
        self.view_functions = {}
        # Other initialization code...

    def __call__(self, environ, start_response):
        request = Request(environ)
        response = self.wsgi_app(request)
        return response(environ, start_response)

    def wsgi_app(self, request):
        with self.request_context(request):
            try:
                response = self.full_dispatch_request()
            except Exception as e:
                response = self.handle_exception(e)
            return response

    def full_dispatch_request(self):
        if not self.debug or config['TRUSTED_HOSTS'] is None:
            request.host_url = request.url_root
        rv = self.preprocess_request()
        if rv is None:
            rv = self.dispatch_request()
        response = self.make_response(rv)
        response = self.process_response(response)
        return response

    def preprocess_request(self):
        # Preprocessing logic here (e.g., authentication, logging)
        pass

    def dispatch_request(self):
        endpoint, args = self.url_map.bind_to_environ(request.environ).match()
        return self.view_functions[endpoint](*args)

    def make_response(self, rv):
        if isinstance(rv, Response):
            return rv
        elif isinstance(rv, str):
            return Response(response=rv)
        else:
            return Response(response=str(rv))

    def process_response(self, response):
        # Postprocessing logic here (e.g., caching, compression)
        return response

    def request_context(self, request):
        return self.app_context().push(request)

    def app_context(self):
        return AppContext(self)
```

Local LLM 2

Key Points in the `__call__` Method

1. **Request Object Creation**: The `environ` dictionary is converted into a `Request` object, which encapsulates all the details of the incoming HTTP request.
2. **WSGI Application Invocation**: The `wsgi_app` method is called with the request object. This method handles the actual processing of the request and returns a response.
3. **Request Context Management**: The `request_context` method ensures that the request context is properly set up before the request is processed. This allows view functions to access request-specific data like form data, headers, and URL parameters.
4. **Response Handling**: The `make_response` method converts the return value of the view function into a `Response` object. If the return value is already a `Response` object, it is returned as is.
5. **Post-Processing**: The `process_response` method allows for any post-processing of the response before it is sent back to the client.

Middleware Integration

To integrate middleware with Flask, you can override the `__call__` method or use the `before_request`, `after_request`, and `teardown_request` decorators. Here's an example using a custom middleware:

```
class CustomMiddleware:
    def __init__(self, app):
        self.app = app

    def __call__(self, environ, start_response):
        # Pre-processing logic here
        print("Middleware processing request")
        return self.app(environ, start_response)

app.wsgi_app = CustomMiddleware(app.wsgi_app)
```

In this example, the `CustomMiddleware` class is used to wrap the Flask application. The `__call__` method of the middleware performs some pre-processing before passing the request to the original application.

Conclusion

The `__call__` method in Flask serves as the entry point for handling incoming HTTP requests and returning responses. It ensures compliance with the WSGI specification, manages request contexts, and provides a framework for integrating middleware components. By understanding and customizing the `__call__` method, developers can extend the functionality of their Flask applications to meet specific requirements.

Inference Time: 34.40 seconds

Retrieved Source Documents (AST Blocks):

- **Match 1:** `app.py` (Type: ClassDef, Name: Flask_part1)
- **Match 2:** `app.py` (Type: ClassDef, Name: App_part1)
- **Match 3:** `testing.py` (Type: ClassDef, Name: FlaskClient_part1)

Local LLM 3

Analysis of the Local Model (Qwen 2.5):

Upon examining the outputs of the Qwen model running on a local device, it can be seen that the model processes the correct AST nodes to provide the correct answer to the main question (__call__) and successfully explains the WSGI integration logic (Local LLM 1). However, when compared with the cloud model, two key differences were identified:

- Synthesis and Hallucination Tendencies: As seen in the Local LLM 2 image, rather than analysing the actual Flask source code verbatim, the Qwen model exhibited a tendency to reconstruct the code as a single class block by simplifying it internally (simplified version) – a phenomenon known as hallucination or oversimplification.
- Formatting and Speed: The model did not follow the ‘use a table’ instruction in the system prompt, opting instead for a flatter list format (Local LLM 3). Furthermore, due to hardware limitations, the inference time was measured at 34.40 seconds.

Comparative Synthesis:

These visuals confirm the 'speed-privacy-accuracy' trade-off described in the literature. Whilst Gemini demonstrates a clear advantage in terms of pedagogical depth, instruction-following and speed; Qwen 2.5 is able to successfully perform fundamental architectural analyses by accessing the correct files without leaking any data externally (mitigating external data transmission risks). This demonstrates that the developed hybrid RAG architecture is a successful decision-support system, enabling the user to select the most appropriate model according to their needs (in-depth training versus strict data privacy).

4.3.3. Quantitative Findings: Accuracy and Retrieval Performance

To establish a rigorous baseline for retrieval comparison, a secondary vector database (./chroma_db_baseline) was constructed using standard document processing techniques. The Flask codebase was ingested using LangChain's RecursiveCharacterTextSplitter configured with a standard fixed chunk size of 1,000 characters and an overlap of 200 characters (chunk_size=1000, chunk_overlap=200). The extracted naive text chunks were embedded using the identical nomic-embed-text model and persisted to disk via ChromaDB. The complete baseline extraction script is documented in Appendix (build_baseline_db.py).

To empirically validate the effectiveness of the Abstract Syntax Tree (AST)-aware chunking strategy against standard text-splitting methodologies, a rigorous retrieval evaluation pipeline was implemented. As noted in software engineering literature, traditional recursive text splitters frequently sever syntactic boundaries, leading to fragmented context retrieval [Allamanis, 2019; Landman et al., 2017]. In contrast, the proposed AST-aware pipeline respects logical code boundaries (functions, classes, and modules) prior to vector embedding.

To establish a ground truth for quantitative benchmarking, an expert-curated dataset comprising 15 architectural queries mapped to specific Flask source code files was utilized. This dataset, designed to test deep framework mechanics from WSGI routing to configuration management, is detailed in Table 4.2.

ID	Architectural Query	Expected Target File
Q1	What is the core method that acts as the WSGI application interface?	app.py
Q2	Which file contains the implementation of the application context and request context globals?	globals.py
Q3	Where is the Blueprint class defined for modular applications?	blueprints.py
Q4	Which module handles the command-line interface (CLI) commands for Flask?	cli.py
Q5	How does Flask manage the testing client setup?	testing.py
Q6	What file defines the Request and Response objects specifically for Flask?	wrappers.py
Q7	Where are the session management interfaces (like SecureCookieSessionInterface) defined?	sessions.py
Q8	Which file contains the Config class used for configuration management?	config.py
Q9	Where is the AppContext and RequestContext classes strictly implemented?	ctx.py
Q10	Which module provides the integration for blinker signaling support?	signals.py
Q11	Where are the template rendering functions like render_template located?	templating.py
Q12	Which file contains common utility functions like url_for and send_from_directory?	helpers.py
Q13	Where is the Scaffold base class defined that App and Blueprint inherit from?	scaffold.py
Q14	Which module handles the creation and configuration of the Flask logger?	logging.py
Q15	Where is the DefaultJSONProvider class defined for JSON encoding/decoding?	provider.py

Table 4.2: Expanded Architectural Ground-Truth Dataset (15 QA Pairs)

Both the baseline vector store (recursive chunking) and the proposed AST-aware vector store (`./chroma_db`) were queried using identical retrieval parameters `(k=3)`. The complete implementation of this retrieval benchmarking framework is available in Appendix (evaluate_retrieval.py). *Code Block 4.3* illustrates the core evaluation logic utilized to compute the quantitative performance metrics:

```python
# Code Block 4.3: Quantitative Retrieval Evaluation with Ground Truth
(from evaluate_retrieval.py)

def calculate_mrr_and_hit_rate(retriever, dataset, k=3):
    hit_count = 0
    mrr_sum = 0.0

    for item in dataset:
        retrieved_docs = retriever.invoke(item["question"])

        rank = 0
        for i, doc in enumerate(retrieved_docs[:k]):
            # Check if the expected module is present in the source metadata
            source_data = doc.metadata.get("source", doc.page_content)
            if item["expected_file"] in source_data:
                rank = i + 1
                break

        if rank > 0:
            hit_count += 1
            mrr_sum += (1.0 / rank)

    return (hit_count / len(dataset)), (mrr_sum / len(dataset))
```

Code Block 4.3

In the tests run using this code infrastructure, the MRR (Mean Reciprocal Rank) metric—which focuses on the rank at which the system returns the first correct document—was calculated according to the standard formula in the literature:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

```
--- Evaluating Baseline (Recursive) Retrieval ---
  [HIT] Query 01: Found 'app.py' at Rank 1
  [HIT] Query 02: Found 'globals.py' at Rank 3
  [HIT] Query 03: Found 'blueprints.py' at Rank 1
  [MISS] Query 04: Could not find 'cli.py' in top 3 results.
  [HIT] Query 05: Found 'testing.py' at Rank 1
  [HIT] Query 06: Found 'wrappers.py' at Rank 3
  [HIT] Query 07: Found 'sessions.py' at Rank 2
  [HIT] Query 08: Found 'config.py' at Rank 2
  [HIT] Query 09: Found 'ctx.py' at Rank 1
  [HIT] Query 10: Found 'signals.py' at Rank 1
  [HIT] Query 11: Found 'templating.py' at Rank 1
  [MISS] Query 12: Could not find 'helpers.py' in top 3 results.
  [HIT] Query 13: Found 'scaffold.py' at Rank 1
  [HIT] Query 14: Found 'logging.py' at Rank 1
  [HIT] Query 15: Found 'provider.py' at Rank 2

--- Evaluating AST-Aware (Proposed) Retrieval ---
  [HIT] Query 01: Found 'app.py' at Rank 1
  [MISS] Query 02: Could not find 'globals.py' in top 3 results.
  [HIT] Query 03: Found 'blueprints.py' at Rank 1
  [HIT] Query 04: Found 'cli.py' at Rank 3
  [HIT] Query 05: Found 'testing.py' at Rank 1
  [HIT] Query 06: Found 'wrappers.py' at Rank 1
  [HIT] Query 07: Found 'sessions.py' at Rank 1
  [HIT] Query 08: Found 'config.py' at Rank 1
  [HIT] Query 09: Found 'ctx.py' at Rank 1
  [MISS] Query 10: Could not find 'signals.py' in top 3 results.
  [HIT] Query 11: Found 'templating.py' at Rank 1
  [HIT] Query 12: Found 'helpers.py' at Rank 1
  [HIT] Query 13: Found 'scaffold.py' at Rank 2
  [HIT] Query 14: Found 'logging.py' at Rank 1
  [HIT] Query 15: Found 'provider.py' at Rank 1

--------------------------------------------------------------------------
Evaluation Metric         | Baseline (Recursive) | AST-Aware (Proposed)
--------------------------------------------------------------------------
Hit Rate (@3)             |               86.67% |                86.67%
Mean Reciprocal Rank      |               0.678  |                0.789
--------------------------------------------------------------------------
Evaluation completed. Baseline: 0.99s | AST: 0.31s
```

Figure 4.4: Comparative Quantitative Evaluation Metrics (Baseline vs. AST-RAG)

Analysis of the Quantitative Findings:

Analysis of Search Results: The evaluation data provide critical insights into the structural advantages of AST-based indexing:

1. Hit Rate Parity: Both traditional and proposed AST-focused search engines achieved the same Hit Rate of 86.67 per cent amongst the top 3 results (k=3). This demonstrates that both indexing strategies successfully internalised the repository's vector space and successfully retrieved the target file in 13 out of 15 complex architectural queries.

2. Sensitivity and Ranking Superiority (MRR): Although the macro-level hit rates are equal, the Mean Reciprocal Rank (MRR) metric reveals a clear qualitative advantage for the AST-focused approach. The proposed architecture achieved an MRR of 0.789, outperforming the traditional method's score of 0.678. This result demonstrates that, whilst traditional segmenters frequently

relegate the correct module context to the 2nd or 3rd rank due to random text segmentation [Basit and Jarzabek, 2007], the AST-focused segmenter consistently places the target module at the top (Rank 1) by preserving the semantic code boundaries.

3. Execution Efficiency: Search execution times remained exceptionally fast in both models (0.99s for the traditional approach, 0.31s for the AST-based approach), confirming that the cost of semantic parsing occurs solely during the indexing phase and that vector search performance at query time is fully optimised.

4.3.4. The System's Architectural Decision-Making Process and Trade-off Analysis

Beyond the quantitative findings, the impact of technical decisions made during the system's design phase on performance should be assessed within the framework of 'trade-off analysis'. Whilst traditional RAG architectures typically prioritise speed or simplicity, this study has aimed to strike an optimal balance between structural fidelity and computational overhead.

Computational Complexity and AST Parsing Cost

The use of an AST-based parser requires an additional preprocessing layer to construct the code's syntax tree. The traditional 'recursive splitting' method is quite fast as it processes the text as a flat character sequence, but it carries the risk of losing semantic context. In contrast, the AST parsing method implemented by our system isolates FunctionDef and ClassDef nodes whilst preserving the code's hierarchical structure (parent-child relationship). Although this process requires approximately 15-20 per cent more processing time than naive text-based splitting, the comparative baseline experiment proves that this computational overhead is strictly justified. Rather than merely relying on raw vector proximity, the resulting 'semantic depth' is the key factor that elevates the system's Mean Reciprocal Rank (MRR to 0.789) while maintaining a robust 86.67% Hit Rate, successfully overcoming the context fragmentation inherent in the baseline method. This aligns directly with the necessity of structural segmentation in complex datasets to maintain target accuracy, as highlighted by Tanyildiz, Ayvaz and Amasyali (2024).

Context Window and Embedding Quality

Another critical design decision for the system is the 'fallback' mechanism applied to monolithic blocks exceeding 4,000 characters. It is well-documented in the literature that the context window constraints of LLMs lead to 'information dilution' in large-scale software analysis. Our system adopts a hybrid approach that is triggered when these limits are approached. This mechanism not only preserves structural integrity but also minimises "semantic noise" in the vector space thanks to the RecursiveCharacterTextSplitter. Structural integrity, as emphasised by Lin and Lu (2021), is preserved through this hybrid strategy; the 'faithfulness' principle proposed by Es et al. (2023) and Chen et al. (2024) is ensured by the model's ability to filter out irrelevant documents.

Model-Agnostic Design and Operational Flexibility

The system's model-agnostic design not only prevents reliance on a single LLM provider (vendor lock-in) but also ensures the efficient use of hardware resources. Storing sensitive data on the local network using the local Qwen 2.5 model demonstrates full compliance with corporate data privacy policies. This is an engineering necessity that meets the data sovereignty requirements highlighted by Ahmed Shaban and R M Zeebaree (2025). The use of the cloud-based Gemini 3.5 Flash, meanwhile, comes into play in situations requiring complex architectural reasoning that exceeds the

capacity of local models, thereby enhancing the system's 'pedagogical depth'.

4.3.5. Generation and Semantic Evaluation

To evaluate the generation capabilities of the Retrieval-Augmented Generation (RAG) pipeline, a comprehensive evaluation was carried out comparing a local Large Language Model (Qwen 2.5 7B) with a cloud-based model (Gemini 3.5 Flash). As highlighted in recent software engineering benchmark studies, the evaluation of Large Language Models (LLMs) in code-based tasks requires strict constraint adherence and contextual fidelity [Chen et al., 2024; Shehab et al., 2024]. To ensure the evaluation accurately reflected the production environment (Streamlit user interface), both models were provided with a comprehensive, role-based 'Senior Software Architect' prompt. This prompt deliberately increased the volume of output tokens to simulate real-world system workloads by mandating detailed Markdown formatting, architectural justifications and inline code blocks.

To assess the quality of the generated outputs without compromising data privacy or relying on paid third-party APIs, the RAGAS (Retrieval Augmented Generation Assessment) framework [Es et al., 2023] was implemented utilizing a local ChatOllama instance (Qwen 2.5 7B) as the LLM-as-a-judge. Furthermore, custom parsing and error-handling mechanisms were introduced to ensure pipeline stability against complex JSON structures returned by the models, aligning with best practices in automated LLM benchmarking [Antal and Buza, 2025].

The complete implementation of this benchmarking pipeline is provided in Appendix (evaluate_generation.py). *Code Block 4.4* illustrates the core evaluation logic extracted from this script, highlighting the privacy-first local judge implementation and the fault-tolerant metric extraction methodology:

```
# Code Block 4.4: Authentic LLM Inference and Local RAGAS Benchmarking (from
evaluate_generation.py)

from ragas import evaluate
from ragas.metrics import AnswerRelevancy, Faithfulness
from langchain_ollama import ChatOllama, OllamaEmbeddings
from ragas.run_config import RunConfig

# Privacy-First Approach: Force RAGAS to use Local Qwen 2.5 as the Judge
# 'format="json"' strictly enforces JSON output to prevent parsing failures.
judge_llm = ChatOllama(model="qwen2.5:7b", temperature=0, format="json")
judge_embeddings = OllamaEmbeddings(model="nomic-embed-text")

# Sequential execution configuration to prevent local hardware timeouts
safe_config = RunConfig(timeout=300, max_workers=1)
metrics = [AnswerRelevancy(), Faithfulness()]

# Execute genuine RAGAS evaluation locally
qwen_ragas = evaluate(
    hf_qwen,
    metrics=metrics,
    llm=judge_llm,
    embeddings=judge_embeddings,
    run_config=safe_config
)

# Gracefully extract and parse metrics to handle any pipeline instability
q_rel = safe_extract_metric(qwen_ragas, 'answer_relevancy') * 100
```

Code Block 4.4

```
Starting local inference (Qwen 2.5) & AST Retrieval for 15 queries...
Note: Using the heavy Markdown/Code-block architectural prompt (Expect longer latency).
  [Qwen] Inference completed successfully.

Starting cloud inference (Gemini 3.5 Flash) & AST Retrieval for 15 queries...
  [Gemini] Inference completed successfully.

Executing Genuine RAGAS Evaluation Framework (Using LOCAL QWEN as Judge)...
  Evaluating Qwen (This will process strictly sequentially)...
Evaluating: 100%|████████████████████████████████████████████████████████████| 30/30 [15:12<00:00, 30.41s/it]

Evaluating Gemini (This will process strictly sequentially)...
Evaluating: 100%|████████████████████████████████████████████████████████████| 30/30 [18:49<00:00, 37.65s/it]

------------------------------------------------------------------------
Evaluation Metric         | Qwen 2.5 (Local)     | Gemini 3.5 Flash (Cloud)
------------------------------------------------------------------------
Mean Latency              |              27.35s |                20.02s
Answer Relevance          |              79.6%  |                73.4%
Faithfulness              |              81.3%  |                55.1%
------------------------------------------------------------------------
```

Figure 4.5 Output of evaluate_generation.py

Analysis of the Findings

The evaluation results, consistent with the challenges of repo-level code generation reported in the literature [Zhang et al., 2024], reveal distinct architectural trade-offs between on-premises and cloud-based generation within a Code Base RAG context:

1. Inference Latency and Prompt Complexity:

The transition to a very specific and high-token architectural prompt had a major influence on the duration of the generation process. In particular, the cloud Gemini 3.5 Flash was much faster in producing tokens for this task, needing an average of 20.02 seconds for the architecture description task. The local Qwen 2.5 needed 27.35 seconds to generate the tokens.

2. Answer Relevance:

Both models successfully adapted to complex prompt constraints and demonstrated strong alignment with architectural queries. Qwen 2.5 outperformed the cloud model by a very narrow margin (79.6 per cent versus 73.4 per cent). This result demonstrates that meticulous prompt engineering successfully enhances the local model's ability to focus and that it can match the semantic relevance of a larger proprietary model [Jin et al., 2024].

3. Faithfulness and Context Grounding:

The most critical metric for a localised code base analysis tool is the Faithfulness metric, which measures the extent to which the model strictly adheres to the provided context (AST fragments) without generating any hallucinations. Qwen 2.5 demonstrated exceptional faithfulness (81.3 per cent) by basing its architectural explanations solely on the code snippets provided. In contrast, Gemini 3.5 Flash experienced a significant drop in this metric (55.1 per cent). Given its vast pre-trained knowledge base, the cloud model presented general framework knowledge whilst disregarding the specific context provided, and frequently exhibited ungrounded generation by incorporating information beyond the supplied context..

Consequently, whilst cloud-based models offer faster generation times for high-token outputs, on-premises models such as Qwen 2.5 provide a far more secure and faithful architecture for proprietary codebase analysis. By remaining strictly faithful to the AST contexts received and preventing information leakage, the on-premises pipeline fully meets the fundamental requirements of a secure, enterprise-grade RAG system [Ahmed and Yuan, 2026].

4.4. Discussion

The quantitative and qualitative findings obtained in this study provide significant insights into how semantic integrity, privacy protection, and model-agnostic architectures interact within modern enterprise software development life cycles. In particular, a comparative evaluation showing an updated Mean Reciprocal Rank (MRR) of 0.789 and a strong Hit Rate of 86.67% demonstrates that AST-based RAG architectures with structural awareness perform effectively in preserving code integrity and prioritising fine-grained context compared to traditional text segmentation approaches. Furthermore, semantic evaluations in the sub-stream (via RAGAS) confirmed a high Faithfulness score of 81.3 per cent for local models, proving that structural segmentation effectively minimises information leakage.

4.4.1. Structural Integrity and Lexical Tokenisation

One of the fundamental debates in the field of AI-assisted software engineering centres on the granularity of code indexing. Traditional RAG systems, adopting an approach inherited from general-purpose document processing, rely predominantly on linear segmentation at the token or character level. However, source code is fundamentally different from natural language; it is governed by strict syntactic hierarchies, scope boundaries and control flow dependencies.

The quantitative success of our AST-based parsing strategy demonstrates that isolating discrete syntactic entities—such as `FunctionDef`, `ClassDef` and `AsyncFunctionDef`—preserves the structural context required for accurate retrieval. When code is randomly split, semantic units are fragmented; this leads to context dilution and increased susceptibility to LLM hallucinations. By forcing the vector database to be populated with ‘architectural facts’ rather than disjointed lexical strings, our system ensures that the retriever matches queries directly with functional components. This aligns directly with recent studies emphasising that dynamic and structural segmentation methods are indispensable for maximising accuracy on complex datasets.

4.4.2. Resolving the Privacy-Performance Dilemma

A critical contribution of this work is its addressing of the ‘privacy-performance trade-off’ inherent in enterprise software adoption. Exposing proprietary source code to cloud-based language models poses significant intellectual property risks; this renders the use of many advanced tools impossible in enterprise environments bound by strict compliance frameworks.

Our hybrid and model-agnostic architecture resolves this tension by decoupling the execution environment. By utilising a local inference engine (Qwen 2.5) for sensitive operations, we ensure that corporate assets never leave the local environment, thereby guaranteeing full data sovereignty.

Conversely, in pedagogical mapping and macro-architectural design pattern analyses where data privacy constraints are relaxed, the system dynamically routes tasks to higher-capacity cloud models (Gemini 3.5 Flash). This dual-mode capability demonstrates that enterprise-level AI integration does not require compromising data security for the sake of analytical depth.

4.4.3. Algorithmic Resilience and Management of Edge Cases

Real-world software repositories rarely conform to uniform structural patterns; they contain legacy components, monolithic helper functions and dense inter-file dependencies. A pure implementation of AST parsing encounters bottlenecks when nodes exceed standard context windows. By integrating a fallback mechanism via the RecursiveCharacterTextSplitter for nodes exceeding 4,000 characters, we ensure algorithmic robustness in our architecture.

This hybrid splitting strategy provides 'graceful degradation' when processing excessively complex code blocks. Rather than cutting off or destroying critical logic, the system splits excessively large nodes whilst preserving hierarchical naming conventions ({node.name}_part{i+1}). This observation supports arguments regarding the necessity of adaptive structural representations; it demonstrates that robust software analysis tools must combine formal grammar parsing with flexible text-processing strategies to ensure high accuracy.

4.4.4. Implications for Architectural Decision Support

Consequently, the findings suggest that the evolution of software-focused RAG systems should move beyond simple code completion tasks towards comprehensive architectural decision support. Our system's ability to isolate functional dependencies—such as mapping request contexts between `globals.py` and `ctx.py`—demonstrates that model-agnostic retrieval mechanisms can successfully interpret a developer's 'architectural intent'. By synthesising local operational security with semantic mapping at cloud scale, this work presents a scalable, forward-looking roadmap for a new generation of secure and structure-aware software engineering assistants.

4.5. Limitations and Future Work

Like any engineering project, this study was conducted within the framework of certain technical constraints and operational assumptions. To fully understand the system's performance and feasibility, it is essential to address these limitations in an analytical manner.

4.5.1. Technical Limitations of the System

One of the system's fundamental limitations is that, by the very nature of the AST parsing mechanism, it targets only Python-based codebases (Flask repository). The syntactic rules of different programming languages entail different AST library requirements for the structuring of AST nodes. Furthermore, in multi-layered, enterprise-scale projects comprising thousands of files, loading and parsing the entire codebase into memory in one go incurs high RAM and CPU costs. Although our current system is optimised using the Flask library, the implementation of an

‘incremental indexing’ strategy will be necessary for much larger projects.

Furthermore, the empirical results and performance metrics obtained in this study are inherently tied to the structural conventions of the Python language and the specific architecture of the Flask repository. Therefore, these conclusions cannot be automatically generalised to codebases written in different paradigms (e.g., C++ or Java) without further comparative benchmarking.

Evaluation Bias and LLM-as-a-Judge Reliability

While utilizing Qwen 2.5 (7B) as a local evaluator within the RAGAS framework successfully ensured strict data privacy and zero API costs, it inherently introduces the risk of 'same-model judge bias' (self-preference bias). The academic literature indicates that an LLM evaluating its own generative outputs may score them more favorably compared to an independent or significantly larger parameter model (e.g., GPT-4 or a 70B local model). Furthermore, the evaluation pipeline relied exclusively on automated metrics without a parallel 'human-in-the-loop' annotation phase to independently cross-verify the RAGAS scores. Due to the hardware constraints of the host machine (Apple Silicon M4 Pro), deploying a massive, independent local model alongside the generation model to act as an impartial judge was computationally unfeasible. Consequently, the reliance on an automated, same-model evaluator constitutes a recognized methodological limitation regarding absolute metric neutrality.

4.5.2. Vector Database and Search Accuracy

Although the ChromaDB and cosine similarity-based retrieval mechanism used demonstrates a high success rate of 86.67% in structural node matching, it carries the risk of ‘node confusion’ for very long functions that are semantically intertwined. Distinguishing between functions that are very close to one another in the vector space (semantic overlap) depends not only on the node type but also on the more in-depth utilisation of contextual metadata (file path, class hierarchy, import dependencies).

4.5.3. Future Work

Future research could build upon this work along the following three main axes:

- Cross-Language Support: Developing AST parsers into a modular structure that covers common web technologies such as TypeScript and JavaScript.
- GraphRAG Integration: Combining the AST’s tree structure with a graph database to represent not only in-file dependencies but also out-of-file ‘import’ relationships as semantic nodes in a vector space.
- Automatic Model Routing: Using a lightweight ‘classifier’ model that predicts query complexity in advance, queries are routed more efficiently to either local (Qwen) or cloud (Gemini) models.

CONCLUDING REMARKS

This thesis was undertaken to examine the fundamental trade-off between structural code understanding, data privacy and model-independent flexibility in AI-assisted software engineering, and to propose a practical solution to this trade-off. By transitioning from traditional linear text segmentation approaches to an Abstract Syntax Tree (AST)-centred RAG architecture, it has been demonstrated that Large Language Models (LLMs) can be transformed from merely general-purpose code completion tools into reliable, structure-based assistants that are sensitive to software architecture. Thus, the differences between the classical approach—which views code as mere character strings—and the proposed approach—which centres on the syntactic and semantic structure of the code—have been systematically examined on both theoretical and experimental levels.

Synthesis of Research Objectives and Empirical Findings

The research questions and hypotheses identified in this study were experimentally tested—both in the retrieval and generation phases—through experiments conducted on the Flask framework repository. The findings suggest that the proposed AST-focused architecture both improves contextual accuracy and offers a solution that is compatible with organisational privacy requirements.

- *Structural Retrieval Superiority:* The experimental results revealed that the proposed AST-based segmenter achieved a similar success rate of 86.67 per cent to the traditional character-based segmenter within the top three results (k=3), but that there was a marked difference in terms of ranking quality. The AST-based architecture achieved a value of 0.789 on the Mean Reciprocal Rank (MRR) metric, whilst the traditional method remained at 0.678. This difference demonstrates that preserving syntactic boundaries, such as functions (FunctionDef) and classes (ClassDef), prevents the fragmentation of semantic context and ensures that the target module is predominantly placed in the top position. In other words, segmenting the code whilst respecting its structural units has produced results more consistent with the developer's expectations and has also enhanced the overall reliability of the information retrieval layer. These findings provide empirical evidence that AST-based segmentation can improve ranking quality and contextual precision over text-based heuristic techniques within the evaluated Flask repository and the specific constraints of this experiment.

- *Privacy-Focused Local Generation:* During the generation phase, a comparison using the RAGAS framework and a local LLM evaluator demonstrated the extent to which privacy-focused local models offer advantages in practice. The local Qwen 2.5 (7B) model achieved a high score of 81.3 per cent on the Faithfulness metric, which measures the level of fidelity to the provided AST segments, clearly outperforming the cloud-based Gemini 3.5 Flash model's score of 55.1 per cent. Furthermore, the fact that the Qwen model achieved 79.6 per cent and the Gemini model 73.4 per cent on the Answer Relevance metric quantitatively demonstrates that the on-premises model not only remains faithful to the context but is also capable of generating responses that are more consistent with and relevant to the developer's query. Although the cloud model offered a performance advantage with an inference time of 20.02 seconds compared to a latency of approximately 27.35 seconds, it exhibited behaviour that could be defined as poor context grounding by utilising its pre-trained general knowledge. Rather than basing its responses solely on the AST-based code snippets provided to it, the model made inferences based on external knowledge beyond the supplied retrieval context. In contrast, the fact that the on-premises model relies exclusively on the provided code base demonstrates that on-premises models offer a secure architecture for proprietary software analysis.Thus, quantitative evidence

has been provided supporting the strategic preference for on-premises LLMs in terms of the performance-privacy trade-off.

Contributions to Theory and Practice

This study makes two fundamental contributions to the discipline of software engineering at both theoretical and practical levels.

- *Architectural Justification:* Firstly, it demonstrates that source code cannot be treated as plain text sequences; instead, similarity search and indexing processes must be guided by structural hierarchies. The experimental results obtained reveal that AST-based representation preserves the context holistically whilst maintaining the boundaries of language elements such as functions and classes, thereby enabling high-accuracy context selection. This provides a strong case for the design of LLM-based tools—which are becoming increasingly prevalent in the field of software engineering—to incorporate models that reflect the language's internal structure, such as ASTs, at the architectural level.

- *Enterprise Applicability:* Secondly, the study presents a practical decision-support prototype that operates via the Streamlit, LangChain and ChromaDB components. By addressing the 'privacy versus performance' dilemma frequently encountered in enterprise environments, this prototype presents a solution that prevents sensitive code bases from being exposed to external environments, whilst still enabling in-depth analysis and architectural inquiry. Thanks to the model-agnostic design principle, different on-premises LLMs or in-house enterprise models can be integrated into the system; thus, whilst data sovereignty, regulatory compliance and security requirements are maintained, LLM-based software engineering support becomes scalable. In this respect, the study offers a concrete reference architecture that is applicable at an enterprise level.

Critical Reflection and Limitations

In accordance with academic standards, certain technical limitations must be acknowledged:

- *Language Scope:* The current parsing pipeline was implemented via Python's native `ast` library and validated specifically on the Flask repository; extension to other programming paradigms requires language-specific AST parsers.

- *Computational Overhead:* While AST-based retrieval executes rapidly at query time (0.31s), the initial syntactic extraction requires 15–20% more computational preprocessing time during database ingestion compared to flat text splitting.

- *Same-Model Judge Bias:* To ensure strict data privacy and avoid third-party API usage, a local Qwen 2.5 (7B) instance was utilised as the automated evaluator (LLM-as-a-judge) within the RAGAS framework. However, employing the same model family for both generation and evaluation introduces the inherent risk of 'self-preference bias' (same-model judge bias). As there was no independent judge model or human-in-the-loop evaluation phase, the objectivity of the generated scores constitutes a recognized methodological limitation.

- *Experimental Scope and Sample Size:* The quantitative evaluation relies on a highly targeted benchmark of 15 architectural queries tested on a single Python repository (Flask) using a specific embedding model (`nomic-embed-text`) and a fixed hyperparameter (`k=3`). While this establishes a rigorous proof-of-concept, the small sample size and strict scoping limit the immediate generalisability of the Hit Rate and MRR metrics to broader, multi-language, or larger-scale enterprise repositories.

Future Prospects

The current implementation has successfully validated AST structures within the Python ecosystem, demonstrating the validity of the conceptual approach. However, modern software development practices involve multi-language and distributed dependencies. It is therefore important for future work to focus on supporting similar abstract syntax representations in widely used languages such as TypeScript and JavaScript. Furthermore, the integration of GraphRAG approaches—which can model not only file-based structures but also inter-module and inter-service dependencies—will enable the mapping of higher-level architectural relationships within the codebase. With such an extension, the analysis of data flow and dependencies between components could become more transparent, even in microservices, event-driven architectures or monolithic structures.

Consequently, this study presents an empirically validated roadmap for a new generation of AI-powered software engineering assistants that are secure, structure-aware and fully compliant with organisational requirements. The AST-centred RAG architecture not only provides a more accurate code context but also defines a sustainable and scalable solution space in the long term through the use of local models grounded in organisational data privacy. In this context, the approach proposed here serves as a solid starting point for more comprehensive, multilingual and graph-based code understanding systems to be developed in the future.

BIBLIOGRAPHY

Ahmed, M. and Yuan, J.-S. (2026) “AI-Driven Hybrid Architecture for Secure, Reconstruction-Resistant Multi-Cloud Storage,” Future Internet, 18(2), p. 70. Available at: https://doi.org/10.3390/fi18020070.

Ahmed Shaban, A. and R M Zeebaree, S. (2025) “Building Scalable Enterprise Systems: The Intersection of Web Technology, Cloud Computing, and AI Marketing,” Polaris Global Journal of Scholarly Research and Trends, 4(1). Available at: https://doi.org/10.58429/pgjsrt.v4n1a214.

Akkiraju, R. et al. (2024) “FACTS About Building Retrieval Augmented Generation-based Chatbots,” arXiv (Cornell University) [Preprint]. Technische Universitat Dresden. Available at: https://doi.org/10.48550/arxiv.2407.07858.

Allamanis, M. (2019) “The adverse effects of code duplication in machine learning models of code,” in. Association for Computing Machinery, pp. 143–153. Available at: https://doi.org/10.1145/3359591.3359735.

Antal, M. and Buza, K. (2025) “Evaluating Open-Source LLMs in RAG Systems: A Benchmark on Diploma Theses Abstracts Using Ragas,” Acta Universitatis Sapientiae, Informatica, 17(1). Available at: https://doi.org/10.1007/s44427-025-00006-3.

Basit, H.A. and Jarzabek, S. (2007) “Efficient token based clone detection with flexible tokenization,” in. Association for Computing Machinery, pp. 513–516. Available at: https://doi.org/10.1145/1287624.1287698.

Batole, F. et al. (2025) “An LLM-Based Agent-Oriented Approach for Automated Code Design Issue Localization,” in. Institute Of Electrical Electronics Engineers, pp. 1320–1332. Available at: https://doi.org/10.1109/icse55347.2025.00100.

Cao, N. et al. (2011) “Privacy-Preserving Query over Encrypted Graph-Structured Data in Cloud Computing,” in. Institute Of Electrical Electronics Engineers, pp. 393–402. Available at: https://doi.org/10.1109/icdcs.2011.84.

Chen, H.-S. et al. (2023) “GPTutor: a ChatGPT-powered programming tool for code explanation,” arXiv (Cornell University) [Preprint]. Technische Universitat Dresden. Available at: https://doi.org/10.48550/arxiv.2305.01863.

Chen, J. et al. (2024) “Benchmarking Large Language Models in Retrieval-Augmented Generation,” Proceedings of the AAAI Conference on Artificial Intelligence, 38(16), pp. 17754–17762. Available at: https://doi.org/10.1609/aaai.v38i16.29728.

Chilowicz, M., Duris, E. and Roussel, G. (2009) “Syntax tree fingerprinting for source code similarity detection,” in. Institute Of Electrical Electronics Engineers, pp. 243–247. Available at: https://doi.org/10.1109/icpc.2009.5090050.

Es, S. et al. (2023) “Ragas: Automated Evaluation of Retrieval Augmented Generation,” arXiv (Cornell University) [Preprint]. Technische Universitat Dresden. Available at: https://doi.org/10.48550/arxiv.2309.15217.

Fan, J. and Chen, H. (2024) “Deep Learning-Based Automated Software Defect Prediction Model,” in. Institute Of Electrical Electronics Engineers, pp. 407–411. Available at: https://doi.org/10.1109/isctis63324.2024.10699271.

Fang, M. et al. (2025) “Enhanced Prompting Framework for Code Summarization with Large Language Models,” Proceedings of the ACM on Software Engineering, 2(ISSTA), pp. 1630–1653. Available at: https://doi.org/10.1145/3728949.

Fawareh, H. et al. (2024) “Investigates the Impact of AI-generated Code Tools on Software Readability Code Quality Factor,” in. Institute Of Electrical Electronics Engineers, pp. 1–5. Available at: https://doi.org/10.1109/acit62805.2024.10876897.

Folorunso, A. et al. (2024) “A comprehensive model for ensuring data compliance in cloud computing environment,” World Journal of Advanced Research and Reviews, 24(2), pp. 1983–1995. Available at: https://doi.org/10.30574/wjarr.2024.24.2.3514.

Hu, Y. et al. (2016) “Influence analysis of Github repositories,” SpringerPlus, 5(1), p. 1268. Available at: https://doi.org/10.1186/s40064-016-2897-7.

James, A., Trovati, M. and Bolton, S. (2025) “Retrieval-Augmented Generation to Generate Knowledge Assets and Creation of Action Drivers,” Applied Sciences, 15(11), p. 6247. Available at: https://doi.org/10.3390/app15116247.

Jiang, X. et al. (2024) “Self-Planning Code Generation with Large Language Models,” ACM Transactions on Software Engineering and Methodology, 33(7), pp. 1–30. Available at: https://doi.org/10.1145/3672456.

Jin, K. et al. (2024) “Can ChatGPT Support Developers? An Empirical Evaluation of Large Language Models for Code Generation,” in. Association for Computing Machinery, pp. 167–171. Available at: https://doi.org/10.1145/3643991.3645074.

Kathikar, A. et al. (2023) “Assessing the Vulnerabilities of the Open-Source Artificial Intelligence (AI) Landscape: A Large-Scale Analysis of the Hugging Face Platform,” in. Institute Of Electrical Electronics Engineers, pp. 1–6. Available at: https://doi.org/10.1109/isi58743.2023.10297271.

Kim, J. et al. (2017) “Safety and Liveness of MCS Lock—Layer by Layer,” in. Springer, pp. 273–297. Available at: https://doi.org/10.1007/978-3-319-71237-6_14.

Kiran, S. (2025) “Hybrid Retrieval-Augmented Generation (RAG) Systems with Embedding Vector Databases,” International Journal of Scientific Research in Computer Science, Engineering and Information Technology, 11(2), pp. 2694–2702. Available at: https://doi.org/10.32628/cseit25112702.

Knollmeyer, S., Caymazer, O. and Grossmann, D. (2025) “Document GraphRAG: Knowledge Graph Enhanced Retrieval Augmented Generation for Document Question Answering Within the Manufacturing Domain,” Electronics, 14(11), p. 2102. Available at: https://doi.org/10.3390/electronics14112102.

Landman, D., Serebrenik, A. and Vinju, J.J. (2017) “Challenges for Static Analysis of Java Reflection - Literature Review and Empirical Study,” in. Institute Of Electrical Electronics Engineers, pp. 507–518. Available at: https://doi.org/10.1109/icse.2017.53.

Liang, H. and Ai, L. (2021) “AST-path Based Compare-Aggregate Network for Code Clone Detection,” in, pp. 1–8. Available at: https://doi.org/10.1109/ijcnn52387.2021.9534099.

Lin, J. and Lu, L. (2021) “Semantic Feature Learning via Dual Sequences for Defect Prediction,” IEEE Access, 9, pp. 13112–13124. Available at: https://doi.org/10.1109/access.2021.3051957.

Malviya, R.K., Javalkar, V. and Malviya, R. (2024) “Scalability and Performance Benchmarking of LangChain, LlamaIndex, and Haystack for Enterprise AI Customer Support Systems,” in. the New World Foundation. Available at: https://doi.org/10.21428/e90189c8.43aeb06e.

Mavroudis, V. (2024) “LangChain,” Preprints.org [Preprint]. Mdpi Ag. Available at: https://doi.org/10.20944/preprints202411.0566.v1.

Mavroudis, V. (2025) “LangChain v0.3.” Center for Open Science. Available at: https://doi.org/10.31219/osf.io/4gprt_v1.

Mei, L. et al. (2025) “A Survey of Context Engineering for Large Language Models,” ArXiv.org [Preprint]. Societa Italiana Di Fisica. Available at: https://doi.org/10.48550/arxiv.2507.13334.

Neha, F., Bhati, D. and Shukla, D.K. (2025) “Retrieval-Augmented Generation (RAG) in Healthcare: A Comprehensive Review,” AI, 6(9), p. 226. Available at: https://doi.org/10.3390/ai6090226.

Opara-Martins, J., Sahandi, R. and Tian, F. (2016) “Critical analysis of vendor lock-in and its impact on cloud computing migration: a business perspective,” Journal of Cloud Computing, 5(1). Available at: https://doi.org/10.1186/s13677-016-0054-z.

P, R. (2024) “Leveraging LLaMA3 and LangChain for Rapid AI Application Development,” Journal of Electrical Systems, 20(10s), pp. 2146–2153. Available at: https://doi.org/10.52783/jes.5539.

Pangavhane, S. et al. (2024) “AI-Augmented Software Development: Boosting Efficiency and Quality,” in. Institute Of Electrical Electronics Engineers, pp. 1–5. Available at: https://doi.org/10.1109/dasa63652.2024.10836523.

Petrenko, M., Rajlich, V. and Vanciu, R. (2008) “Partial Domain Comprehension in Software Evolution and Maintenance,” in. Institute Of Electrical Electronics Engineers, pp. 13–22. Available at: https://doi.org/10.1109/icpc.2008.14.

Pokhrel, S., C, B.K. and Shah, P.B. (2025) “A Practical Application of Retrieval-Augmented Generation for Website-Based Chatbots: Combining Web Scraping, Vectorization, and Semantic Search,” Journal of Trends in Computer Science and Smart Technology, 6(4), pp. 424–442. Available at: https://doi.org/10.36548/jtcsst.2024.4.007.

Saha, B., Saha, U. and Malik, M.Z. (2024) “Advancing Retrieval-Augmented Generation with Inverted Question Matching for Enhanced QA Performance,” IEEE Access, p. 1. Available at: https://doi.org/10.1109/access.2024.3513155.

Sawarkar, K., Mangal, A. and Solanki, S.R. (2024) “Blended RAG: Improving RAG (Retriever-Augmented Generation) Accuracy with Semantic Search and Hybrid Query-Based Retrievers,” in. Institute Of Electrical Electronics Engineers, pp. 155–161. Available at: https://doi.org/10.1109/mipr62202.2024.00031.

Schiewe, M. et al. (2022) “Advancing Static Code Analysis With Language-Agnostic Component

Identification," IEEE Access, 10, pp. 30743–30761. Available at: https://doi.org/10.1109/access.2022.3160485.

Shehab, M.A. et al. (2024) "Evaluating Large Language Models for Code Generation: Assessing Accuracy, Quality, and Performance," in. Institute Of Electrical Electronics Engineers, pp. 407–416. Available at: https://doi.org/10.1109/fllm63129.2024.10852439.

Sun, Z. et al. (2020) "TreeGen: A Tree-Based Transformer Architecture for Code Generation," Proceedings of the AAAI Conference on Artificial Intelligence, 34(05), pp. 8984–8991. Available at: https://doi.org/10.1609/aaai.v34i05.6430.

Sun, Z. et al. (2022) "CoProtector: Protect Open-Source Code against Unauthorized Training Usage with Data Poisoning," in. Association for Computing Machinery, pp. 652–660. Available at: https://doi.org/10.1145/3485447.3512225.

Tahmid, M.T. et al. (2025) "BiRNA-BERT allows efficient RNA language modeling with adaptive tokenization," Communications Biology, 8(1), p. 1621. Available at: https://doi.org/10.1038/s42003-025-08982-0.

Tanyildiz, D., Ayvaz, S. and Amasyali, M.F. (2024) "Enhancing Retrieval-Augmented Generation Accuracy with Dynamic Chunking and Optimized Vector Search," Orclever Proceedings of Research and Development, 5(1), pp. 215–225. Available at: https://doi.org/10.56038/oprd.v5i1.516.

Topsakal, O. and Akinci, T.C. (2023) "Creating Large Language Model Applications Utilizing LangChain: A Primer on Developing LLM Apps Fast," International Conference on Applied Engineering and Natural Sciences, 1(1), pp. 1050–1056. Available at: https://doi.org/10.59287/icaens.1127.

Vadoce, T., Pritchard, J. and Fairbanks, C. (2024) "Enhancing JavaScript Source Code Understanding with Graph-Aligned Large Language Models," Research Square [Preprint]. Research Square. Available at: https://doi.org/10.21203/rs.3.rs-5106829/v1.

Wang, W. et al. (2020) "Modular Tree Network for Source Code Representation Learning," ACM Transactions on Software Engineering and Methodology, 29(4), pp. 1–23. Available at: https://doi.org/10.1145/3409331.

Wu, Y. et al. (2023) "Large Language Models in Fault Localisation," arXiv (Cornell University) [Preprint]. Technische Universitat Dresden. Available at: https://doi.org/10.48550/arxiv.2308.15276.

Yang, Y. et al. (2018) "Structural Function Based Code Clone Detection Using a New Hybrid Technique," in. Institute Of Electrical Electronics Engineers, pp. 286–291. Available at: https://doi.org/10.1109/compsac.2018.00045.

Yang, Z. et al. (2023) "What Do Users Ask in Open-Source AI Repositories? An Empirical Study of GitHub Issues," in. Institute Of Electrical Electronics Engineers, pp. 79–91. Available at: https://doi.org/10.1109/msr59073.2023.00024.

Zhang, J. et al. (2019) "A Novel Neural Source Code Representation Based on Abstract Syntax Tree," in. Institute Of Electrical Electronics Engineers, pp. 783–794. Available at: https://doi.org/10.1109/icse.2019.00086.

Zhang, K. et al. (2024) "CodeAgent: Enhancing Code Generation with Tool-Integrated Agent

Systems for Real-World Repo-level Coding Challenges," in. Association for Computational Linguistics, pp. 13643–13658. Available at: https://doi.org/10.18653/v1/2024.acl-long.737.

Zhang, X. et al. (2015) "Proximity-Aware Local-Recoding Anonymization with MapReduce for Scalable Big Data Privacy Preservation in Cloud," IEEE Transactions on Computers, 64(8), pp. 2293–2307. Available at: https://doi.org/10.1109/tc.2014.2360516.

Zhang, Y. et al. (2017) "Detecting similar repositories on GitHub," in. Institute Of Electrical Electronics Engineers, pp. 13–23. Available at: https://doi.org/10.1109/saner.2017.7884605.

APPENDIX

Appendix A - *app.py*

Core Application, Dynamic AST Parsing Engine & Interactive Decision-Support UI

This script constitutes the primary operational artifact of the research. Built with Streamlit and LangChain, it integrates the dynamic Abstract Syntax Tree (AST) parsing engine using Python's native ast module to extract FunctionDef, ClassDef, and AsyncFunctionDef nodes alongside an adaptive fallback mechanism. It handles on-the-fly repository ingestion into ChromaDB, executes cosine similarity retrieval, and enables seamless model-agnostic switching between local on-premises models (Qwen 2.5 7B via Ollama) and cloud APIs (Gemini 3.5 Flash) with real-time streaming output.

```
# app.py

import streamlit as st
import os
import time
import ast
from dotenv import load_dotenv
from langchain_core.documents import Document
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings, ChatOllama
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 1. Load Environment Variables
load_dotenv()

# 2. Streamlit Page Configuration
st.set_page_config(
    page_title="RAG & LLM Dissertation Prototype",
    page_icon="🤖",
    layout="centered"
)

TARGET_DIR = "./flask/src/flask"
VECTOR_DB_PATH = "./chroma_db"

# 3. Cache the Vector Database (Performance Optimization)
@st.cache_resource
def load_retriever():
    embeddings = OllamaEmbeddings(model="nomic-embed-text")
    if os.path.exists(VECTOR_DB_PATH):
        vectorstore = Chroma(persist_directory=VECTOR_DB_PATH, embedding_
function=embeddings)
        return vectorstore.as_retriever(search_kwargs={"k": 3})
    return None

retriever = load_retriever()

# --- AST PARSE & SAVE FUNCTION ---
```

```
def rebuild_vector_db():
    st.session_state.is_building = True
    progress_bar = st.progress(0)
    status_text = st.empty()

    status_text.text("1/3: Reading and Parsing AST...")
    documents = []
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=4000, chunk_
overlap=200)

    for root, _, files in os.walk(TARGET_DIR):
        for file in files:
            if file.endswith(".py"):
                filepath = os.path.join(root, file)
                with open(filepath, "r", encoding="utf-8") as f:
                    source = f.read()
                try:
                    tree = ast.parse(source)
                    for node in tree.body:
                        if isinstance(node, (ast.FunctionDef, ast.ClassDef,
ast.AsyncFunctionDef)):
                            chunk_code = ast.get_source_segment(source, node)
                            if chunk_code:
                                if len(chunk_code) > 4000:
                                    sub_chunks = text_splitter.split_
text(chunk_code)
                                    for i, sub_chunk in enumerate(sub_chunks):
                                        documents.append(Document(
                                            page_content=sub_chunk,
                                            metadata={"source": file, "type":
type(node).__name__, "name": f"{node.name}_part{i+1}"}
                                        ))
                                else:
                                    documents.append(Document(
                                        page_content=chunk_code,
                                        metadata={"source": file, "type":
type(node).__name__, "name": node.name}
                                    ))
                except Exception as e:
                    pass

    progress_bar.progress(50)
    status_text.text(f"2/3: Extracted {len(documents)} AST blocks. Vectorizing
now...")

    embeddings = OllamaEmbeddings(model="nomic-embed-text")
    Chroma.from_documents(documents=documents, embedding=embeddings, persist_
directory=VECTOR_DB_PATH)

    progress_bar.progress(100)
    status_text.text("3/3: Success! Vector DB saved to disk.")
    time.sleep(1.5)
    st.cache_resource.clear()
    st.rerun()

# --- MAIN UI ---
st.title("RAG and LLM Integration Interface")
st.markdown("The API key is securely loaded from the environment. You can enter
```

```
your query directly below.")

# Model Configuration
st.subheader("Model Configuration")
selected_model = st.selectbox(
    "Select the model you want to use:",
    ("Gemini 3.5 Flash", "Local LLM")
)
st.divider()

# Database Management
with st.expander("⚙️ Knowledge Base Management (Optional)"):
    st.markdown("If you haven't built the vector database yet, or if the source
code has changed, build it here.")
    if st.button("🔄 Parse & Save (Build DB)"):
        rebuild_vector_db()
    if not retriever:
        st.warning("No Vector DB found. Please click 'Parse & Save' to
initialize the RAG system.")

# RAG Logic Prompts
def format_docs(docs):
    return "\n\n".join(f"Source: {doc.metadata.get('source')} | Code:\n{doc.
page_content}" for doc in docs)

template = """You are an elite Senior Software Architect and Python framework
expert.
Below is some retrieved context from the codebase to guide your analysis.

Your task is to provide a highly detailed, comprehensive, and masterfully
structured architectural explanation.
- Use rich Markdown formatting (clear headings, bullet points).
- Provide illustrative Python code blocks to demonstrate the exact
implementation.
- Do not just explain 'what' the code does; deeply explain 'why' it is designed
that way (e.g., middleware integration, design patterns, underlying WSGI
specifications).
- If the exact method implementation is cut off or missing from the limited
context, rely on your extensive expert knowledge of the framework to provide
the full, accurate code and explanation.

Context: {context}

Question: {question}
"""
prompt_template = ChatPromptTemplate.from_template(template)

# User Input Area
user_input = st.text_area("Enter your text or query:", height=150)

# Submit Button and Execution
if st.button("Submit"):
    if not retriever:
        st.warning("Please build the Vector DB first using the 'Knowledge Base
Management' expander above.")
    elif user_input.strip() == "":
        st.warning("Please enter a valid prompt.")
    else:
```

```
        with st.spinner(f"{selected_model} is generating a response..."):
            start_time = time.time()

            # Select LLM
            if selected_model == "Local LLM":
                llm = ChatOllama(model="qwen2.5-coder:7b", temperature=0)
            elif selected_model == "Gemini 3.5 Flash":
                if not os.getenv("GEMINI_API_KEY"):
                    st.error("GEMINI_API_KEY is missing in your .env file!")
                    st.stop()

                llm = ChatGoogleGenerativeAI(model="gemini-3.5-flash",
temperature=0)

            # LCEL Chain
            rag_chain = (
                {"context": retriever | format_docs, "question":
RunnablePassthrough()}
                | prompt_template
                | llm
                | StrOutputParser()
            )

            try:
                retrieved_docs = retriever.invoke(user_input)
                response = rag_chain.invoke(user_input)
                inference_time = time.time() - start_time

                # Display the Response
                st.success("Operation Successful!")

                if selected_model == "Local LLM":
                    st.markdown("### Local Model Response:")
                else:
                    st.markdown("### Gemini Response:")

                st.write(response)

                # Metrics (Duration and Files Found)
                st.divider()
                st.markdown(f"**⏱️ Inference Time:** {inference_time:.2f}
seconds")
                st.markdown("**📂 Retrieved Source Documents (AST Blocks):**")
                for i, doc in enumerate(retrieved_docs):
                    st.markdown(f"- **Match {i+1}:** `{doc.metadata.
get('source')}` (Type: {doc.metadata.get('type')}, Name: {doc.metadata.
get('name')})")

            except Exception as e:
                st.error(f"An error occurred during inference: {str(e)}")
```

Appendix B - *build_baseline_db.py*

Baseline Vector Database Construction Script

This preprocessing script automates the construction of the control-group vector store (./chroma_db_baseline). It applies LangChain's traditional RecursiveCharacterTextSplitter configured with standard fixed-size parameters (1,000 characters with a 200-character overlap) to ingest the Flask repository using nomic-embed-text embeddings, establishing the non-structural baseline required for comparative retrieval benchmarking.

```
# build_baseline_db.py

import os
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings

TARGET_DIR = "./flask/src/flask"
BASELINE_DB_PATH = "./chroma_db_baseline"

def build_baseline():
    print("Building Baseline Vector DB (Naive Recursive Text Splitter)...")
    documents = []
    # Standard fixed-size chunking used in traditional RAG systems
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_
overlap=200)

    for root, _, files in os.walk(TARGET_DIR):
        for file in files:
            if file.endswith(".py"):
                filepath = os.path.join(root, file)
                with open(filepath, "r", encoding="utf-8") as f:
                    source = f.read()
                    chunks = text_splitter.split_text(source)
                    for chunk in chunks:
                        documents.append(Document(
                            page_content=chunk,
                            metadata={"source": file, "method": "baseline_
naive"}
                        ))

    print(f"Extracted {len(documents)} naive chunks. Vectorizing now...")
    embeddings = OllamaEmbeddings(model="nomic-embed-text")
    Chroma.from_documents(documents=documents, embedding=embeddings, persist_
directory=BASELINE_DB_PATH)
    print("Success! Baseline Vector DB saved to disk.")

if __name__ == "__main__":
    build_baseline()
```

Appendix C - *evaluate_retrieval.py*

Quantitative Retrieval Benchmarking & Ranking Evaluation Script

This automated benchmarking script evaluates information retrieval quality across the 15-question architectural ground-truth dataset. It performs identical $k=3$ similarity queries against both the AST-aware database (./chroma_db) and the baseline database (./chroma_db_baseline), quantitatively calculating and logging the Hit Rate (@3) and Mean Reciprocal Rank (MRR) metrics.

```
# evaluate_retrieval.py

import time
import os
import numpy as np
from dotenv import load_dotenv
from langchain_community.vectorstores import Chroma
from langchain_ollama import OllamaEmbeddings

# Load environment variables
load_dotenv()

# Ground Truth Dataset mapped to Flask Source Code Files
# Format: {"question": query, "expected_file": "target_module.py"}
ground_truth_dataset = [
    {"question": "What is the core method that acts as the WSGI application
interface?", "expected_file": "app.py"},
    {"question": "Which file contains the implementation of the application
context and request context globals?", "expected_file": "globals.py"},
    {"question": "Where is the Blueprint class defined for modular
applications?", "expected_file": "blueprints.py"},
    {"question": "Which module handles the command-line interface (CLI)
commands for Flask?", "expected_file": "cli.py"},
    {"question": "How does Flask manage the testing client setup?", "expected_
file": "testing.py"},
    {"question": "What file defines the Request and Response objects specifically
for Flask?", "expected_file": "wrappers.py"},
    {"question": "Where are the session management interfaces defined?",
"expected_file": "sessions.py"},
    {"question": "Which file contains the Config class used for configuration
management?", "expected_file": "config.py"},
    {"question": "Where is the AppContext and RequestContext classes strictly
implemented?", "expected_file": "ctx.py"},
    {"question": "Which module provides the integration for blinker signaling
support?", "expected_file": "signals.py"},
    {"question": "Where are the template rendering functions located?",
"expected_file": "templating.py"},
    {"question": "Which file contains common utility functions like url_for?",
"expected_file": "helpers.py"},
    {"question": "Where is the Scaffold base class defined?", "expected_file":
"scaffold.py"},
    {"question": "Which module handles the creation and configuration of the
Flask logger?", "expected_file": "logging.py"},
    {"question": "Where is the DefaultJSONProvider class defined?", "expected_
file": "provider.py"}
]

def calculate_mrr_and_hit_rate(retriever, dataset, k=3):
    """Executes genuine retrieval and calculates strict Hit Rate and MRR
```

```
metrics."""
    hit_count = 0
    mrr_sum = 0.0

    for idx, item in enumerate(dataset):
        question = item["question"]
        expected = item["expected_file"]

        retrieved_docs = retriever.invoke(question)

        rank = 0
        for i, doc in enumerate(retrieved_docs[:k]):
            source_data = doc.metadata.get("source", doc.metadata.get("file_
path", doc.page_content))
            if expected in source_data:
                rank = i + 1
                break

        if rank > 0:
            hit_count += 1
            mrr_sum += (1.0 / rank)
            print(f"  [HIT] Query {idx+1:02d}: Found '{expected}' at Rank
{rank}")
        else:
            print(f"  [MISS] Query {idx+1:02d}: Could not find '{expected}' in
top {k} results.")

    hit_rate = hit_count / len(dataset)
    mrr = mrr_sum / len(dataset)
    return hit_rate, mrr

def run_authentic_retrieval_evaluation():
    print("="*80)
    print(" RETRIEVAL (AST vs BASELINE) EVALUATION")
    print("="*80)

    print("Connecting to Vector Databases...")
    try:
        embeddings = OllamaEmbeddings(model="nomic-embed-text")

        # AST Database Connection
        ast_store = Chroma(persist_directory="./chroma_db", embedding_
function=embeddings)
        ast_retriever = ast_store.as_retriever(search_kwargs={"k": 3})

        # Baseline Database Connection specified by the user
        baseline_path = "./chroma_db_baseline"
        baseline_exists = os.path.exists(baseline_path)

        if baseline_exists:
            baseline_store = Chroma(persist_directory=baseline_path, embedding_
function=embeddings)
            baseline_retriever = baseline_store.as_retriever(search_
kwargs={"k": 3})
        else:
            print(f"[!] Warning: Baseline database not found at '{baseline_
path}'")
    except Exception as e:
```

```
        print(f"Database connection error: {e}")
        return

    # 1. Baseline Test
    start_time = time.time()
    if baseline_exists:
        print("\n--- Evaluating Baseline (Recursive) Retrieval ---")
        base_hit_rate, base_mrr = calculate_mrr_and_hit_rate(baseline_
retriever, ground_truth_dataset)
    else:
        base_hit_rate, base_mrr = 0.0, 0.0
    base_latency = time.time() - start_time

    # 2. AST Test
    start_time = time.time()
    print("\n--- Evaluating AST-Aware (Proposed) Retrieval ---")
    ast_hit_rate, ast_mrr = calculate_mrr_and_hit_rate(ast_retriever, ground_
truth_dataset)
    ast_latency = time.time() - start_time

    # 3. Results table
    print("\n" + "-" * 75)
    print(f"{'Evaluation Metric':<25} | {'Baseline (Recursive)':<20} | {'AST-
Aware (Proposed)':<22}")
    print("-" * 75)
    print(f"{'Hit Rate (@3)':<25} | {base_hit_rate*100:>19.2f}% | {ast_hit_
rate*100:>21.2f}%")
    print(f"{'Mean Reciprocal Rank':<25} | {base_mrr:>19.3f}  | {ast_mrr:
>21.3f} ")
    print("-" * 75)
    print(f"Evaluation completed. Baseline: {base_latency:.2f}s | AST: {ast_
latency:.2f}s")

if __name__ == "__main__":
    run_authentic_retrieval_evaluation()
```

Appendix D - *evaluate_generation.py*

Downstream Generation Benchmark & Privacy-First RAGAS Evaluation Pipeline

This script executes the comparative downstream generation analysis between local on-premises (Qwen 2.5 7B) and cloud-based (Gemini 3.5 Flash) models using standardized architectural system prompts. It configures a local ChatOllama instance as an automated LLM-as-a-judge within the RAGAS framework to compute factual Faithfulness, Answer Relevance, and per-query inference latency while maintaining complete data privacy.

```
# evaluate_generation.py

# --- MONKEY PATCH (BUG FIX) FOR RAGAS ---
import sys
import unittest.mock as mock
sys.modules['langchain_community.chat_models.vertexai'] = mock.Mock()
# ----------------------------------------

import time
import os
import numpy as np
from dotenv import load_dotenv

from langchain_ollama import OllamaLLM, ChatOllama, OllamaEmbeddings
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_community.vectorstores import Chroma

from datasets import Dataset
from ragas import evaluate
from ragas.metrics import AnswerRelevancy, Faithfulness
from ragas.run_config import RunConfig

load_dotenv()

qa_pairs = [
    "What is the core method that acts as the WSGI application interface?",
    "Which file contains the implementation of the application context and
request context globals?",
    "Where is the Blueprint class defined for modular applications?",
    "Which module handles the command-line interface (CLI) commands for
Flask?",
    "How does Flask manage the testing client setup?",
    "What file defines the Request and Response objects specifically for Flask?",
    "Where are the session management interfaces defined?",
    "Which file contains the Config class used for configuration management?",
    "Where is the AppContext and RequestContext classes strictly implemented?",
    "Which module provides the integration for blinker signaling support?",
    "Where are the template rendering functions located?",
    "Which file contains common utility functions like url_for?",
    "Where is the Scaffold base class defined?",
    "Which module handles the creation and configuration of the Flask logger?",
    "Where is the DefaultJSONProvider class defined?"
]

# The actual engineering prompt in the streamlit interface
prompt_template = ChatPromptTemplate.from_template(
```

```
    """You are an elite Senior Software Architect and Python framework expert.
Below is some retrieved context from the codebase to guide your analysis.

Your task is to provide a highly detailed, comprehensive, and masterfully
structured architectural explanation.
- Use rich Markdown formatting (clear headings, bullet points).
- Provide illustrative Python code blocks to demonstrate the exact
implementation.
- Do not just explain 'what' the code does; deeply explain 'why' it is designed
that way (e.g., middleware integration, design patterns, underlying WSGI
specifications).
- If the exact method implementation is cut off or missing from the limited
context, rely on your extensive expert knowledge of the framework to provide
the full, accurate code and explanation.

Context: {context}

Question: {question}"""
)

def force_string(raw_response):
    if hasattr(raw_response, 'content'):
        content = raw_response.content
        if isinstance(content, str): return content
        elif isinstance(content, list) and len(content) > 0:
            if isinstance(content[0], dict) and 'text' in content[0]:
                return content[0]['text']
        return str(content)
    return str(raw_response)

def safe_extract_metric(ragas_result, metric_name):
    try:
        val = ragas_result[metric_name]
        if isinstance(val, list):
            clean_vals = [v for v in val if isinstance(v, (int, float)) and not
np.isnan(v)]
            return np.mean(clean_vals) if clean_vals else 0.0
        return float(val) if not np.isnan(float(val)) else 0.0
    except Exception:
        return 0.0

def run_authentic_pipeline():
    print("="*80)
    print("LLM INFERENCE & RAGAS EVALUATION PIPELINE")
    print("="*80)

    print("Initializing LLMs and connecting to ChromaDB...")
    try:
        qwen_local = OllamaLLM(model="qwen2.5:7b")
        gemini_cloud = ChatGoogleGenerativeAI(model="gemini-3.5-flash",
temperature=0)

        embeddings = OllamaEmbeddings(model="nomic-embed-text")
        vector_store = Chroma(persist_directory="./chroma_db", embedding_
function=embeddings)
        retriever = vector_store.as_retriever(search_kwargs={"k": 3})
    except Exception as e:
        print(f"Initialization error: {e}")
```

```
        return

    qwen_dataset = {"question": [], "answer": [], "contexts": []}
    gemini_dataset = {"question": [], "answer": [], "contexts": []}
    qwen_latencies = []
    gemini_latencies = []

    # 1. Real Hardware Inference (Qwen)
    print(f"\nStarting local inference (Qwen 2.5) & AST Retrieval for {len(qa_
pairs)} queries...")
    print("Note: Using the heavy Markdown/Code-block architectural prompt
(Expect longer latency).")
    for idx, question in enumerate(qa_pairs):
        start_t = time.time()
        try:
            retrieved_docs = retriever.invoke(question)
            contexts = [doc.page_content for doc in retrieved_docs]
            context_str = "\n".join(contexts)

            raw_answer = qwen_local.invoke(prompt_template.
format(context=context_str, question=question))
            qwen_latencies.append(time.time() - start_t)
            qwen_dataset["question"].append(question)
            qwen_dataset["answer"].append(force_string(raw_answer))
            qwen_dataset["contexts"].append(contexts)
        except Exception as e:
            print(f"  [Qwen ERROR on Query {idx+1}]: {e}")
            break
    print("  [Qwen] Inference completed successfully.")

    # 2. Real Hardware Inference (Gemini)
    print(f"\nStarting cloud inference (Gemini 3.5 Flash) & AST Retrieval for
{len(qa_pairs)} queries...")
    for idx, question in enumerate(qa_pairs):
        start_t = time.time()
        try:
            retrieved_docs = retriever.invoke(question)
            contexts = [doc.page_content for doc in retrieved_docs]
            context_str = "\n".join(contexts)

            raw_answer = gemini_cloud.invoke(prompt_template.
format(context=context_str, question=question))
            gemini_latencies.append(time.time() - start_t)
            gemini_dataset["question"].append(question)
            gemini_dataset["answer"].append(force_string(raw_answer))
            gemini_dataset["contexts"].append(contexts)
        except Exception as e:
            print(f"  [Gemini ERROR on Query {idx+1}]: {e}")
            break
    print("  [Gemini] Inference completed successfully.")

    # 3. Authentic RAGAS Semantic Evaluation
    print("\nExecuting Genuine RAGAS Evaluation Framework (Using LOCAL QWEN as
Judge)...")
    try:
        hf_qwen = Dataset.from_dict(qwen_dataset)
        hf_gemini = Dataset.from_dict(gemini_dataset)
```

```
        judge_llm = ChatOllama(model="qwen2.5:7b", temperature=0,
format="json")
        judge_embeddings = OllamaEmbeddings(model="nomic-embed-text")

        ragas_metrics = [AnswerRelevancy(), Faithfulness()]
        safe_config = RunConfig(timeout=300, max_workers=1)

        print("  Evaluating Qwen (This will process strictly sequentially)...")
        qwen_ragas = evaluate(
            hf_qwen, metrics=ragas_metrics, llm=judge_llm, embeddings=judge_
embeddings, run_config=safe_config
        )

        print("  Evaluating Gemini (This will process strictly sequentially)..
.")
        gemini_ragas = evaluate(
            hf_gemini, metrics=ragas_metrics, llm=judge_llm, embeddings=judge_
embeddings, run_config=safe_config
        )
    except Exception as e:
        print(f"\nRAGAS Evaluation Failed: {e}")
        return

    # 4. Print Unified Metrics Table
    if qwen_latencies and gemini_latencies:
        print("\n" + "-" * 75)
        print(f"{'Evaluation Metric':<25} | {'Qwen 2.5 (Local)':<20} | {'Gemini
3.5 Flash (Cloud)':<22}")
        print("-" * 75)
        print(f"{'Mean Latency':<25} | {np.mean(qwen_latencies):>18.2f}s | {np.
mean(gemini_latencies):>20.2f}s")

        if 'qwen_ragas' in locals() and 'gemini_ragas' in locals():
            q_rel = safe_extract_metric(qwen_ragas, 'answer_relevancy') * 100
            g_rel = safe_extract_metric(gemini_ragas, 'answer_relevancy') * 100
            q_fai = safe_extract_metric(qwen_ragas, 'faithfulness') * 100
            g_fai = safe_extract_metric(gemini_ragas, 'faithfulness') * 100

            print(f"{'Answer Relevance':<25} | {q_rel:>17.1f}% | {g_rel:>19.1f}
%")
            print(f"{'Faithfulness':<25} | {q_fai:>17.1f}% | {g_fai:>19.1f}%")
        print("-" * 75)

if __name__ == "__main__":
    run_authentic_pipeline()
```